Benchmarking and Profiling MLPerf Workloads on GPUs - llama3.1-8b

A Dissertation as a Course requirement for

Master of Science in Mathematics

Sai Saketh Cherukuri

24010203008

SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING

(Deemed to be University)

Department of Mathematics and Computer Science

Prashanthi Nilayam Campus

March - 2026

SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING
(Deemed to be University)

Dept. of **Mathematics & Computer Science**
Prasanthi Nilayam Campus

CERTIFICATE

This is to certify that this Dissertation titled **Benchmarking and Profiling MLPerf Workloads on GPUs - llama3.1-8b** submitted by Sai Saketh Cherukuri, 24010203008, Department of Mathematics and Computer Science, Prashanthi Nilayam Campus is a bonafide record of the original work done under my supervision as a Course requirement for the Degree of Master of Science in Mathematics.

.....
Dr. R Raghunatha Sarma
Dissertation Supervisor

Countersigned by

Place: Puttaparthi

.....
Prof. N Uday Kiran
Head of the Department

Date: March 28, 2026

Vidyagiri, Prasanthi Nilayam - 515 134, Andhra Pradesh, India
Tel: +91 8555 287235 | hoddmacs@sssihl.edu.in | www.sssihl.edu.in

DECLARATION

The Dissertation titled **Benchmarking and Profiling MLPerf Workloads on GPUs - llama3.1-8b** was carried out by me under the supervision of Dr. R Raghunatha Sarma, Department of Mathematics and Computer Science, Prashanthi Nilayam Campus as a Course requirement for the Degree of Masters of Science in Mathematics and has not formed the basis for the award of any degree, diploma or any other such title by this or any other University.

Saisaketh

……..

Place : Puttaparthi

Sai Saketh Cherukuri
24010203008
II M.Sc Mathematics
Prashanthi Nilayam Campus

Date : March 28, 2026

Acknowledgements

I am deeply grateful for the opportunity to have worked on this project over the past several months. This journey has broadened my understanding of a field I am truly passionate about and has shaped me both academically and personally.

First and foremost, I express my profound gratitude to Bhagawan Sri Sathya Sai Baba, my eternal source of strength and inspiration. His timeless teachings have been my guiding light throughout this endeavour.

I express my sincere gratitude to my guide, Dr. R. Raghunatha Sarma, for his unwavering support, patient guidance, and encouragement. His insightful feedback and dedication have been instrumental in shaping the direction and quality of this work.

I extend my heartfelt thanks to Mr. M. Naveen (AI Performance Engineer, RedHat), for their invaluable mentorship and industry perspectives. Their guidance has greatly enriched both my understanding and this work.

I sincerely acknowledge the researchers and authors whose published works form the foundation of this study. Their contributions to the field of large language model serving and inference optimisation have provided the intellectual scaffolding upon which this dissertation is built.

I sincerely thank Dr. N. Uday Kiran and all the faculty of the Department of Mathematics and Computer Science, SSSIHL, for their continued support, encouragement, and the resources made available to me throughout this project.

Finally, I wholeheartedly thank my parents, friends, and well-wishers, whose unconditional love and encouragement have sustained me through every challenge along the way.

Benchmarking and Profiling MLPerf workloads on GPUs - llama3.1-8b

Sai Saketh Cherukuri

Abstract

This report presents a systematic benchmarking study of Meta's Llama 3.1-8B on a single NVIDIA H200 NVL GPU using MLPerf Inference v6.0 and vLLM v0.17.1. The central question is practical: how much additional serving capacity can be extracted from a single GPU through software configuration alone, without acquiring new hardware or compromising output quality?

We conduct a single-variable parameter study across 19 vLLM engine knobs, a stacked optimisation sweep, a feature-toggle study, and GPU execution trace analysis using NVIDIA Nsight Systems. FP8 weight quantisation delivers **+34.5%** throughput by halving model memory from 16 GB to 8 GB, directly reducing the per-token HBM bandwidth demand that bottlenecks autoregressive decode. Scaling batch size from 16 to 1024 adds a further **+205%** over the BF16 baseline, shifting the GPU from memory-bandwidth-bound to compute-bound operation. Together, these two changes triple serving capacity on identical hardware, raising throughput from 8.51 to 25.95 samples/sec, with ROUGE-1 remaining at 38.84, only 0.04 points above the BF16 reference and well within MLPerf compliance.

Nsight Systems profiling reveals the structural cause: at batch size 16, the GPU sits idle for 1,122 ms between batches, wasting 50% of all GPU time on CPU-side dispatch overhead. At batch size 1024, this gap collapses to 0.516 ms. Right-sizing the KV context window to 2,668 tokens is a prerequisite that unlocks all high-throughput configurations. Large-batch serving also compresses the P90-to-P99.9 latency tail from 156 s to 82 s, producing substantially more predictable behaviour for production SLA compliance. These results demonstrate that commodity configuration changes to a standard vLLM deployment are sufficient to triple GPU utilisation at no additional hardware cost.

Keywords: MLPerf Inference, Llama3.1-8B, Transformers, vLLM, GPU benchmarking, Nsight Systems, GPU profiling

Table of Contents

1 Introduction **5**
1.1 Motivation and Industry Context 5
1.2 Contributions and Scope 5
1.3 Report Structure 6

2 Literature Review **7**

3 Large Language Models and the LLaMA Family **9**
3.1 What Are Large Language Models? 9
3.2 Emergent Abilities and the Case for Benchmarking 10
3.3 LLM Inference: Prefill and Decode 10
3.4 The LLaMA Model Series 11
3.5 Why Llama 3.1-8B for MLPerf? 13

4 LLM Inference Runtimes **14**
4.1 Challenges of LLM Serving 14
4.2 vLLM: Design, Features, and Usage in This Work 14
4.3 Competing Runtimes 19

5 MLPerf Inference Framework **20**
5.1 Scenarios 20
5.2 Divisions and Categories 20
5.3 LoadGen 22
5.4 Compliance and Reproducibility 22

6 Transformer Architecture and Llama3.1-8B **23**
6.1 Origins and Motivation 23
6.2 Input Representation: Tokenisation, Embeddings, and Positional Encoding 24
6.3 Scaled Dot-Product Attention 25
6.4 Multi-Head Attention 26
6.5 Position-Wise Feed-Forward Sub-Layer 26
6.6 The Encoder Stack 26
6.7 The Decoder Stack 27
6.8 The Complete Encoder-Decoder Model 28
6.9 Architectural Variants: Encoder-Only, Decoder-Only, Encoder-Decoder 29
6.10 Llama3.1-8B Architecture 30

7 Experimental Setup **33**
7.1 Hardware 33
7.2 Software Stack 33
7.3 CNN/DailyMail Summarisation Dataset 33

7.4 Accuracy Targets 35

8 Methodology **36**
8.1 Serving Stack 36
8.2 Experiment Phases 36
8.3 Baseline vLLM Engine Configuration 37
8.4 LoadGen Configuration 37

9 Throughput Results: Single-Variable Sweep **38**
9.1 Throughput Delta Across Configurations 41
9.2 Quantisation and Batch-Size Effects 42
9.3 Throughput vs. Latency Trade-off 43

10 Deep Dive: Prefix Caching, Chunked Prefill, and Quantisation **44**
10.1 Automatic Prefix Caching 44
10.1.1 Mechanism 44
10.1.2 Results and the `prefix_off` Measurement 45
10.2 Chunked Prefill 47
10.2.1 Mechanism 47
10.2.2 Results and Token Budget Tuning 48
10.3 FP8 Quantisation 49
10.3.1 Why LLM Inference Is Memory-Bandwidth-Bound 49
10.3.2 Quantisation Formats: E4M3 vs INT8 vs BF16 50
10.3.3 Results 50
10.3.4 Accuracy Safety of FP8 51
10.3.5 Interaction with Other Optimisations 52
10.3.6 Summary of Recommendations 52

11 Throughput Results: Stacked Optimisation **53**

12 Feature-Toggle Study **57**
12.1 Reproducibility and Production Validation 59
12.2 Production Recommendations 60

13 Critical Findings **61**

14 Accuracy Validation **63**
14.1 Discussion 63

15 Latency Percentile Profiles **66**
15.1 Latency Percentile Profiles Across Configurations 67

16 Batch Size Analysis via Nsight Systems **69**
16.1 Per-Batch GPU Utilisation 69
16.2 Inter-Batch Idle Gap 70
16.3 Kernel Category Breakdown 70
16.4 Decode Step Analysis 71

16.5 Kernel Category Utilisation 72

17 Issues and Solutions **73**
17.1 KV Cache Exhaustion 73
17.2 CUDA Memory Fragmentation 74
17.3 Transformers Library Incompatibility 75
17.4 Features That Hurt Performance 75

18 Future Work **76**

19 Conclusion **78**

1 Introduction

1.1 Motivation and Industry Context

The rapid proliferation of large language models (LLMs) in commercial and research applications has shifted the bottleneck in the machine learning lifecycle from *training* to *inference*. While a frontier model such as GPT-5.4 or Llama 3.1-405B is trained once at enormous cost, it may subsequently serve billions of inference requests over its operational lifetime. Industry analyses estimate that inference accounts for 60–70% of total energy consumption in production ML systems [1], a proportion that continues to grow as LLM-powered products, including chatbots, code assistants, summarisation engines, and retrieval-augmented generation pipelines, scale to millions of concurrent users.

The economic implications are profound. Each token generated during autoregressive decoding requires reading the model's entire weight matrix from GPU high-bandwidth memory (HBM), making LLM inference fundamentally *memory-bandwidth-bound*. For an 8-billion-parameter model stored in BF16, this translates to approximately 16 GB of weight data read per generated token. At production-scale query volumes, even small improvements in inference throughput translate directly into reduced hardware requirements, lower energy consumption, and decreased cost per query. Conversely, inefficient serving configurations waste GPU cycles on padding, memory fragmentation, and synchronisation overhead, squandering the hardware investment.

This economic pressure has driven intense innovation in LLM serving systems. Frameworks such as vLLM [2], TensorRT-LLM [3], and SGLang [4] have introduced techniques such as PagedAttention [2], continuous batching, speculative decoding, and prefix caching that can collectively improve throughput by an order of magnitude over naive implementations. However, these optimisations interact in complex and sometimes counter-intuitive ways: enabling one feature may negate the benefit of another, and the optimal configuration depends heavily on the specific model, hardware, dataset, and workload characteristics. Understanding these interactions systematically requires controlled experimentation under a standardised benchmark framework.

MLPerf Inference [5], maintained by MLCommons, provides exactly this framework. It defines standardised workloads, quality targets, scenario specifications, and compliance procedures that ensure fair, reproducible comparison across submissions. The inclusion of Llama3.1-8B as a benchmark task in MLPerf Inference v5.0 reflects the community's recognition that LLM inference is now a first-class workload demanding the same rigour previously reserved for image classification and object detection benchmarks.

1.2 Contributions and Scope

This report documents the complete process of benchmarking Meta's Llama3.1-8B model on a single NVIDIA H200 NVL GPU using the MLPerf Inference framework. The work spans three dimensions: (i) a systematic parameter sweep to maximise offline

throughput, (ii) accuracy validation using ROUGE-based evaluation on the CNN/DailyMail summarisation task, and (iii) offline latency profiling to characterise end-to-end system behaviour.

The experiments were carried out in three phases. First, a *single-variable study* changed one vLLM engine parameter at a time from a fixed BF16 baseline to measure its individual effect. Second, a *feature-toggle study* started from an already-optimised FP8 baseline (batch size 1024, reduced `max_model_len`) and switched individual features on or off to measure their marginal value. Third, an *accuracy study* swept sampling parameters and output budgets to confirm ROUGE score parity with the reference. Two vLLM versions were used across the study: v0.17.1 for the single-variable and stacked sweeps, and v0.10.0 for the feature-toggle study, reflecting the version available at the time of that experiment. Throughout, we encountered practical challenges, most notably KV cache exhaustion, segmentation faults caused by CUDA memory fragmentation, and the need to pin the HuggingFace Transformers library to a compatible version, all of which are documented in Section 17.

The principal contributions of this work are:

- A comprehensive single-variable characterisation of 19 vLLM engine parameters under vLLM 0.17.1's V1 engine, identifying FP8 quantisation (+34.5% in tokens/s), batch size scaling (+112.7% in tokens/s in isolation; +205% samples/s end-to-end when combined with FP8), and CUDA graph compilation (load-bearing at −20.8% when disabled) as the dominant throughput levers.
- A feature-toggle study under vLLM 0.10.0 revealing that prefix caching delivers the largest single-feature gain (+57.6%) on shared-prefix workloads, while several commonly recommended optimisations (high GPU memory utilisation, multi-step scheduling, scheduler delay) are actively counter-productive.
- An accuracy study demonstrating that FP8 and float16 quantisation are accuracy-safe (absolute ROUGE-1 $\Delta < 0.04$ points from the MLPerf target) while large BF16 batch sizes cause unexpected ROUGE degradation exceeding −32%, an anomaly documented for further investigation.
- Practical guidance on KV cache right-sizing, CUDA memory fragmentation mitigation, and library version pinning for reproducible MLPerf submissions.

1.3 Report Structure

Sections 3–6 provide background on LLMs, the LLaMA model family, the literature, vLLM and competing runtimes, the MLPerf framework, and the transformer architecture. Section 7 and Section 8 detail the experimental environment and methodology. Sections 9–15 present the results in six parts: single-variable throughput sweep, deep dive into the dominant knobs, stacked optimisation sweep, feature-toggle study, accuracy validation, and latency profiling with Nsight Systems GPU trace analysis. Section 13 consolidates the cross-cutting system-level findings that directly inform deployment decisions related to throughput. Section 17 documents practical problems encountered and their solutions, and Sections 18 and 19 outline directions for future work and conclusions.

2 Literature Review

This literature review focuses on the concrete engineering contributions of the vLLM project and the MLPerf/MLCommons benchmarking methodology, together with a brief survey of system-level advances that influenced the experimental design of this work.

vLLM [2] addresses the two practical bottlenecks that dominate large-model serving: (1) KV cache memory pressure and fragmentation, and (2) scheduler and dispatch overhead when serving many concurrent, variable-length requests. The paper introduces PagedAttention [2], a block-based KV cache layout that maps logical token positions to small fixed-size physical blocks (default 16 tokens) via a per-sequence block table. This mapping eliminates the need for contiguous per-sequence allocations, drastically reducing both internal and external fragmentation and enabling copy-on-write prefix sharing, where identical prompt prefixes reuse the same physical blocks until divergence. The result is an order-of-magnitude reduction in effective KV memory waste, directly enabling much larger concurrent batch sizes without swapping to CPU memory.

On the scheduler side, vLLM implements continuous (iteration-level) batching: rather than grouping requests into static batches and waiting for the slowest member to finish, the scheduler admits and retires sequences at each single-token iteration, immediately filling freed slots with pending requests. This algorithm keeps the GPU busy despite large per-request length variance and avoids head-of-line blocking. The vLLM work also explores several practical engineering levers including chunked prefill, async CPU-side scheduling, multi-step scheduling, CUDA graph capture, and multiple quantisation strategies (FP8, AWQ/INT8, GPTQ). This work replicates and systematically evaluates several of these levers under MLPerf constraints, with particular focus on quantisation, batch size scaling, and compilation behaviour.

MLPerf Inference [5, 6] supplies the measurement framework that makes system comparisons reproducible. The LoadGen driver (C++ core with Python bindings) defines four canonical scenarios: SingleStream, MultiStream, Server, and Offline, each with distinct arrival processes and target metrics (latency percentiles or throughput). MLPerf enforces strict compliance rules including minimum run durations, minimum sample counts, deterministic reference outputs for closed-division runs, and per-metric accuracy thresholds. This work uses the MLPerf Llama3.1-8B workload definition (dataset split, pretokenised inputs, ROUGE targets) to ensure results are interpretable in the community context.

Complementary contributions shaped the software choices evaluated here. FlashAttention [15] provides tiled attention kernels that avoid materialising the full $N \times N$ attention matrix in HBM, reducing peak memory usage from $O(N^2)$ to $O(N)$ and enabling efficient attention over long contexts. Speculative decoding [16, 17] uses a smaller draft model to propose multiple candidate tokens that are then verified by the target model in a single pass. This technique is more applicable to latency-sensitive SingleStream and Server scenarios than to the Offline throughput setting evaluated here, and is discussed as a direction for future work.

Hardware-aware optimisations are also central to the literature. Hopper-era FP8 primitives (E4M3 format) enable weight- and activation-level quantisation that halves

model memory and reduces memory bandwidth pressure at negligible quality cost on most workloads. TensorRT-LLM [3] illustrates how vendor-supplied kernel fusion can further amplify these gains for tightly-coupled deployments. This work explicitly tests FP8 and other quantisation modes to measure their empirical throughput and accuracy trade-offs under MLPerf rules.

Taken together, vLLM provides the serving abstractions and implementation patterns tuned and measured in this report, MLPerf supplies the evaluation protocol, and the adjacent literature on attention kernels, quantisation, and speculative decoding supplies the lower-level techniques evaluated or discussed as future directions.

3 Large Language Models and the LLaMA Family

3.1 What Are Large Language Models?

Large language models (LLMs) are neural networks trained on massive text corpora to model the joint probability distribution over token sequences. Given a sequence of tokens $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ drawn from a vocabulary $\mathcal{V}$, the model factorises the joint probability via the chain rule:

$$P(\mathbf{x}) = \prod_{t=1}^{T} P(x_t \mid x_1, x_2, \ldots, x_{t-1}; \theta) \tag{1}$$

where θ denotes the model parameters. This factorisation is exact and requires no conditional independence assumptions. The model is trained to maximise the log-likelihood of the training corpus $\mathcal{D}$, equivalently minimising the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \sum_{t=1}^{T} \log P(x_t \mid x_{<t}; \theta) \tag{2}$$

By pre-training on hundreds of billions to trillions of tokens, LLMs acquire wide linguistic and factual knowledge applicable to downstream tasks via *prompting* alone, without task-specific fine-tuning. This *in-context learning* capability [9] emerged as models scaled beyond a few billion parameters.

The scaling laws of Kaplan et al. [7] showed that model loss decreases roughly as a power law with model size, dataset size, and compute budget, motivating the expansion toward ever-larger models. Specifically, they proposed a unified equation combining the contributions of both model size N and dataset size D:

$$L(N, D) = L_\infty + \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \tag{3}$$

where $\alpha_N \approx 0.076$ and $\alpha_D \approx 0.095$ are the fitted scaling exponents for parameters and data respectively, $N_c \approx 8.8 \times 10^{13}$ and $D_c \approx 5.4 \times 10^{13}$ are the corresponding characteristic scales, and L_∞ is an irreducible loss floor representing the inherent entropy of natural language. This decomposition implies diminishing but persistent returns from scaling either dimension and has been widely used to guide training budget decisions.

Subsequent work by Hoffmann et al. [8] (Chinchilla) refined the practical implications of these scaling laws. They found that for compute-optimal training, model size and training tokens should scale equally: for every doubling of model size, the number of training tokens should also be doubled, yielding an optimal ratio of approximately 20 tokens per parameter [8].

GPT-3 (175B parameters, 2020) [9] demonstrated that at sufficient scale, models could solve few-shot benchmarks with no additional training. However, Chinchilla [8] revealed that GPT-3 was significantly undertrained: its 300B training tokens amount

to fewer than 2 tokens per parameter, far below the compute-optimal ratio of 20. This finding shifted the community toward smaller, well-trained models, exactly the regime where Llama3.1-8B sits, having been trained on approximately 15 trillion tokens [11].

3.2 Emergent Abilities and the Case for Benchmarking

Wei et al. [10] documented *emergent abilities*: capabilities that appear abruptly once a model exceeds a critical scale threshold. Examples include chain-of-thought reasoning, multi-step arithmetic, and code generation, none of which are present in smaller models trained on identical data. These emergent abilities make it essential to benchmark LLMs of different sizes under standardised conditions, as performance on a given task cannot be reliably extrapolated from smaller-scale experiments. The MLPerf Inference framework addresses this need by providing controlled, reproducible evaluation scenarios that hold the task, dataset, and quality targets fixed while varying the system under test.

The practical consequence is that medium-scale models (7B–13B parameters) occupy a critical niche: they are small enough to serve efficiently on a single GPU yet large enough to exhibit useful emergent abilities for tasks such as summarisation, translation, and question answering. Llama3.1-8B exemplifies this niche, and its inclusion in MLPerf Inference v5.0 reflects the benchmark suite's evolution to capture the workloads that dominate real-world LLM deployment.

3.3 LLM Inference: Prefill and Decode

Modern LLMs are decoder-only transformers that generate text autoregressively. Given a prompt of N tokens, the model generates each new token by executing a forward pass over all preceding tokens and sampling the highest-probability next token. This process has two distinct phases:

- **Prefill (prompt processing):** all N prompt tokens are processed in a single parallel forward pass, populating the *KV cache* for each attention layer. Prefill is *compute-bound* and benefits from high matrix-multiply throughput. For a model with L layers, hidden dimension d, and N input tokens, the prefill computation involves approximately $2 \times L \times (12 \times d^2 + 2 \times N \times d)$ FLOPs, dominated by the $O(N \times d^2)$ linear projection cost at each layer.
- **Decode (autoregressive generation):** one new token is generated per forward pass, each time reading all accumulated KV cache entries. Decode is *memory-bandwidth-bound*: for an 8B parameter model in BF16 ($\approx$16 GB), the full weight matrix must be read once per generated token. The arithmetic intensity of decode is approximately $2P \,/\, (P \times B_{\text{param}})$ FLOP/byte, where P is the parameter count and $B_{\text{param}} = 2$ bytes for BF16. For Llama3.1-8B this gives $2 \times 8.03 \times 10^9 \,/\, (8.03 \times 10^9 \times 2) = 1$ FLOP/byte, far below the H200 NVL's compute-to-bandwidth ratio of approximately 412 FLOP/byte at BF16 precision ($\approx$1,979 TFLOPS BF16 divided by 4.8 TB/s).

This prefill/decode duality fundamentally shapes how inference systems are optimised. During prefill, the GPU's tensor cores are well-utilised because large matrix

multiplications on the N input tokens provide sufficient arithmetic intensity. During decode, each step processes only a single new token, so the dominant cost becomes reading the model weights and KV cache from HBM rather than computing attention products. This asymmetry means that prefill throughput scales with GPU compute (TFLOPS), while decode throughput scales with GPU memory bandwidth (TB/s). The NVIDIA H200 NVL provides 4.8 TB/s of HBM3e bandwidth [14], enabling approximately $4.8 \times 10^{12} \,/\, (16.06 \times 10^{9}) \approx 299$ decode tokens per second for a single stream at BF16 precision.

The KV cache grows with each generated token. For Llama3.1-8B with 32 layers, 8 KV heads, and per-head dimension $d_{head} = d_{model}/n_{heads} = 4096/32 = 128$ in BF16, each token occupies:

$$2 \times L \times n_{kv} \times d_{head} \times B_{\text{param}} = 2 \times 32 \times 8 \times 128 \times 2 = 131{,}072 \,\text{bytes} \; (128\,\text{KB}) \quad (4)$$

For a batch of B sequences each up to S tokens, the total KV cache is $B \times S \times 128\,\text{KB}$. At batch size 1024 with the CNN/DailyMail maximum of 2 668 tokens (2 540 input + 128 output), this amounts to:

$$1024 \times 2{,}668 \times 128\,\text{KB} \approx 352\,\text{GB} \quad (5)$$

more than double the H200's 141 GB HBM capacity. Managing KV cache memory efficiently through techniques such as PagedAttention [2], prefix sharing, and FP8 KV cache quantisation is therefore central to achieving high batch-level throughput.

3.4 The LLaMA Model Series

Meta AI released the LLaMA[1] series as open-weight foundation models, enabling research and commercial deployment without access to proprietary APIs. Table 1 summarises the evolution.

Table 1 The LLaMA model family released by Meta AI.

Release	Sizes	Context	Key improvements
LLaMA 1 [12]	7B, 13B, 33B, 65B	2K	First competitive open-weight model; pre-norm (RMSNorm), SwiGLU activation, RoPE positional encoding
LLaMA 2 [13]	7B, 13B, 70B	4K	RLHF-tuned chat variants; GQA on 70B only; context doubled from 2K to 4K
LLaMA 3 [11]	8B, 70B, 405B	8K	Tiktoken tokenizer (128,000 tokens); GQA on all sizes; improved multilingual and code
LLaMA 3.1 [11]	8B, 70B, 405B	**128K**	Extended context via improved RoPE scaling; stronger instruction following; 128K context on all sizes

[1] LLaMA: Large Language Model Meta AI.

LLaMA 1 (February 2023).

The first LLaMA models [12] were released in four sizes (7B, 13B, 33B, and 65B parameters) and showed that a model trained on more tokens, even one with fewer parameters, can match or exceed much larger models (e.g. GPT-3 at 175B) on standard benchmarks. LLaMA 1 introduced pre-normalisation with RMSNorm, SwiGLU activation, and Rotary Positional Embeddings (RoPE). However, its 2,048-token context window limited applicability to long-document tasks.

LLaMA 2 (July 2023).

LLaMA 2 [13] doubled the context window from 2,048 to 4,096 tokens and introduced chat-tuned variants using supervised fine-tuning followed by reinforcement learning from human feedback (RLHF). Grouped-Query Attention (GQA) was adopted on the 70B model to reduce KV cache memory; the 7B and 13B variants retain standard multi-head attention.

LLaMA 3 and 3.1 (April/July 2024).

LLaMA 3 [11] replaced the SentencePiece tokeniser with a Tiktoken-based vocabulary of 128,000 tokens (up from 32,000 in LLaMA 2), substantially improving multilingual coverage and code representation while reducing the average number of tokens needed per piece of text. LLaMA 3.1 extended the context window to 128K tokens via an improved RoPE scaling strategy, enabling document-level reasoning. The 8B model matches LLaMA 2 70B on several benchmarks while being approximately 9× smaller in parameter count.

Category	Benchmark	Llama 3 8B	Gemma 2 9B	Mistral 7B	Llama 3 70B	Mixtral 8x22B	GPT 3.5 Turbo	Llama 3 405B	Nemotron 4 340B	GPT-4 (0125)	GPT-4o	Claude 3.5 Sonnet
General	MMLU (5-shot)	69.4	**72.3**	61.1	**83.6**	76.9	70.7	87.3	82.6	85.1	89.1	**89.9**
	MMLU (0-shot, CoT)	**73.0**	72.3△	60.5	**86.0**	79.9	69.8	88.6	78.7◁	85.4	**88.7**	88.3
	MMLU-Pro (5-shot, CoT)	**48.3**	–	36.9	**66.4**	56.3	49.2	73.3	62.7	64.8	74.0	**77.0**
	IFEval	**80.4**	73.6	57.6	**87.5**	72.7	69.9	**88.6**	85.1	84.3	85.6	88.0
Code	HumanEval (0-shot)	**72.6**	54.3	40.2	**80.5**	75.6	68.0	89.0	73.2	86.6	90.2	**92.0**
	MBPP EvalPlus (0-shot)	**72.8**	71.7	49.5	**86.0**	78.6	82.0	88.6	72.8	83.6	87.8	**90.5**
Math	GSM8K (8-shot, CoT)	**84.5**	76.7	53.2	**95.1**	88.2	81.6	**96.8**	92.3♢	94.2	96.1	96.4♢
	MATH (0-shot, CoT)	**51.9**	44.3	13.0	**68.0**	54.1	43.1	73.8	41.1	64.5	**76.6**	71.1
Reasoning	ARC Challenge (0-shot)	83.4	**87.6**	74.2	**94.8**	88.7	83.7	**96.9**	94.6	96.4	96.7	96.7
	GPQA (0-shot, CoT)	32.8	–	28.8	**46.7**	33.3	30.8	51.1	–	41.4	53.6	**59.4**
Tool use	BFCL	**76.1**	–	60.4	84.8	–	**85.9**	88.5	86.5	88.3	80.5	**90.2**
	Nexus	**38.5**	30.0	24.7	**56.7**	48.5	37.2	**58.7**	–	50.3	56.1	45.7
Long context	ZeroSCROLLS/QuALITY	81.0	–	–	90.5	–	–	**95.2**	–	**95.2**	90.5	90.5
	InfiniteBench/En.MC	65.1	–	–	78.2	–	–	**83.4**	–	72.1	82.5	–
	NIH/Multi-needle	98.8	–	–	97.5	–	–	98.1	–	**100.0**	**100.0**	90.8
Multilingual	MGSM (0-shot, CoT)	**68.9**	53.2	29.9	**86.9**	71.1	51.4	**91.6**	–	85.9	90.5	**91.6**

Fig. 1 LLaMA 3.1 model family performance comparison. The 8B variant (used in this work as the MLPerf workload) achieves strong results across academic benchmarks. Source: Meta AI, The LLaMA 3 Herd of Models [11].

3.5 Why Llama 3.1-8B for MLPerf?

The 8B parameter size occupies a practical sweet spot: small enough to fit on a single GPU (NVIDIA A100 80GB, H100 80GB, or H200 141GB) while capable enough to produce coherent abstractive summaries on CNN/DailyMail. MLCommons adopted Llama3.1-8B as the LLM benchmark task [6] because it is open-weight, broadly reproducible, and representative of how modern instruction-tuned LLMs are deployed in production. Its 128K context window, though not exercised in the CNN/DailyMail task (inputs are $\leq$ 2,540 tokens), demonstrates architectural generality.

4 LLM Inference Runtimes

4.1 Challenges of LLM Serving

Deploying LLMs efficiently at scale poses interconnected challenges that distinguish LLM serving from conventional deep learning inference:

- **Memory-bandwidth-bound decoding.** During autoregressive generation each step processes only one new token, so arithmetic intensity is far below GPU compute peaks. The full model weight matrix must be read from HBM for every generated token, making throughput proportional to memory bandwidth rather than compute.
- **KV cache memory pressure.** Key and value tensors for all layers and sequence positions must reside in GPU memory throughout generation. At tens to hundreds of kilobytes per token per sequence, even moderate batch sizes can saturate GPU HBM.
- **Variable-length batching.** Unlike image classification where all inputs share the same shape, LLM inputs and outputs vary widely in length. Static batching wastes memory through padding; dynamic batching must handle request completions at different times without stalling the GPU.
- **Head-of-line blocking.** In naive serving, a long request occupies GPU resources and blocks shorter requests behind it in the queue. Efficient systems must interleave decode steps from concurrent requests to avoid this.
- **Prefill–decode interference.** Prefill is compute-intensive and can block the GPU for hundreds of milliseconds on long prompts, while decode is memory-bandwidth-intensive and spans many short sequential steps. Mixing the two naively causes GPU utilisation imbalances and increases time-to-first-token for co-located requests.

4.2 vLLM: Design, Features, and Usage in This Work

vLLM [2] is an open-source, high-throughput LLM inference and serving engine originally developed in the Sky Computing Lab at UC Berkeley and released in June 2023, with the core research published at SOSP 2023. It has rapidly become the dominant framework for both research and production LLM serving on NVIDIA GPUs, and is the officially mandated System Under Test (SUT) for the MLCommons Llama3.1-8B benchmark task. vLLM is written in Python with performance-critical paths in C++/CUDA. It ships both an *offline batch API* (`vllm.LLM`) for scripted evaluation and an *online server API* (`vllm serve`) that exposes an OpenAI-compatible REST endpoint.

Architecturally, vLLM separates the *scheduler* (CPU-side logic that decides which requests to execute, block allocations, and token sampling) from the *execution engine* (GPU-side model forward pass). This separation allows the scheduler to be swapped or tuned independently of the model and enables future extensions like preemption-based fairness without changing the GPU execution path.

PagedAttention.

The fundamental bottleneck in LLM serving is KV cache memory management. In the naïve approach used by HuggingFace Transformers, each sequence's KV cache is

allocated as a contiguous block reserved up to the maximum possible sequence length. This causes two types of waste: *internal fragmentation* (tokens never generated within the reserved space) and *external fragmentation* (free GPU memory scattered in unusable gaps between live allocations). The original vLLM paper measured that 60–80% of KV cache memory was wasted through fragmentation in naïve implementations [2].

PagedAttention [2] solves this by borrowing the *paging* concept from operating system virtual memory. Just as an OS maps virtual pages to physical page frames scattered throughout RAM, PagedAttention maps a sequence's *logical* KV cache positions to *physical* blocks scattered throughout GPU HBM. Each physical block holds key and value tensors for a fixed number of token positions (the block size, defaulting to 16 tokens). A per-sequence *block table* serves the same role as a page table in virtual memory. When the attention kernel needs to compute $\text{Attention}(Q, K, V)$ for a given sequence, it uses the block table to gather the correct key and value blocks from their scattered locations.

This design eliminates both fragmentation types. Physical blocks from different sequences can be interleaved anywhere in GPU memory, with no contiguity required. When sequences complete, their blocks are returned to a global free list and immediately available for reuse. By reducing KV cache waste from 60–80% to under 4%, PagedAttention enables the same GPU to serve 2–4× more concurrent sequences, directly multiplying batch-level throughput [2].

An additional benefit is **copy-on-write prefix sharing**: when multiple requests share an identical prompt prefix (e.g. a fixed system instruction), their KV blocks for that prefix can point to the same physical memory. Blocks are only cloned when a sequence produces output diverging from the shared prefix, at which point a copy-on-write mechanism allocates a new physical block and copies the original data. This is the mechanical basis for prefix caching [2].

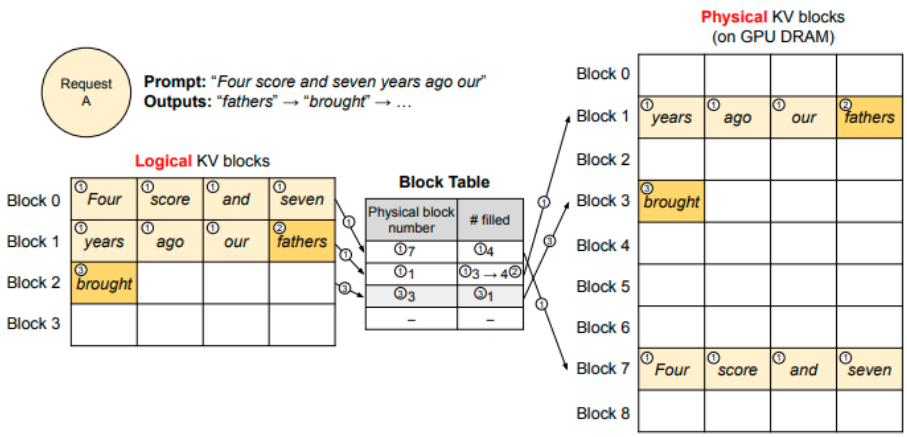

Fig. 2 PagedAttention: logical KV cache blocks are mapped to non-contiguous physical memory pages via a block table, eliminating fragmentation and enabling copy-on-write prefix sharing. Adapted from Kwon et al. [2].

Continuous (iteration-level) batching.

Traditional *static batching* groups a fixed set of requests, runs all of them to completion, and only then starts the next batch. If one request requires 800 tokens and another requires 20, the GPU executes 780 near-idle decode iterations waiting for the long request to finish, wasting compute that could have served new requests.

Continuous batching, originally proposed as Orca [22] and implemented by vLLM at the iteration level, resolves this completely. After each single-token decode step, the scheduler inspects which sequences have finished (by emitting an end-of-sequence token or reaching `max_tokens`) and immediately fills those slots with new pending requests from the waiting queue. The batch composition changes at every iteration, so GPU utilisation stays high regardless of request length heterogeneity.

The scheduler maintains three conceptual queues: *waiting* (requests received but not yet started), *running* (requests with allocated KV cache blocks actively generating tokens), and *swapped* (requests whose KV blocks have been temporarily evicted to CPU memory under memory pressure). At each scheduling step, the scheduler admits as many waiting requests as possible until the KV cache block budget is exhausted, then runs one decode step for all running requests simultaneously. This first-come-first-served (FCFS) policy ensures fairness under uniform request priority [36].

In practice, iteration-level continuous batching achieves 2–6× higher throughput than static batching under realistic mixed-length workloads, since the GPU is never blocked by long-tail sequences while shorter requests accumulate in the queue [2].

CUDA Graph Capture and Compilation.

Each decode step in a transformer model involves hundreds of CUDA kernel launches, each requiring a CPU-to-GPU dispatch via the CUDA driver. For small batch sizes, kernel execution time is short and CPU-side dispatch overhead becomes a significant fraction of per-step latency.

CUDA graph capture records a sequence of GPU operations into a replayable graph object. Once captured, the entire decode step can be replayed with a single CPU-to-GPU command, eliminating per-kernel launch overhead. vLLM's V1 engine implements a multi-level compilation system:

- **O0:** No compilation. Every decode step dispatches kernels eagerly. This mode is useful only for debugging.
- **O1:** Selective CUDA graph capture for the most frequently executed decode paths.
- **O2 (Default):** Full CUDA graph capture combined with `torch.compile` optimisations on the model graph. This is the V1 default.
- **O3:** Adds additional graph optimisation passes beyond O2. Marginal benefit on most workloads.

The V1 engine defaults to O2, and our experiments confirm that this is load-bearing: disabling compilation entirely (O0) causes a −20.8% throughput regression, the largest single negative result in the sweep (exp_04, Section 9). The O2→O3 upgrade yields only +1.4%, suggesting that O2 already captures the vast majority of available optimisation headroom on this workload.

Quantisation.

vLLM supports multiple quantisation methods to reduce weight memory and improve throughput:

- **FP8 (float8_e4m3fn):** uses hardware FP8 GEMM instructions natively available on NVIDIA Hopper (H100/H200, compute capability 9.0). The E4M3 format uses 1 sign bit, 4 exponent bits, and 3 mantissa bits, yielding a dynamic range of ± 448. It reduces model weights from $\approx$16 GB (BF16) to $\approx$8 GB with negligible accuracy loss. This is the primary quantisation scheme used in this work, yielding a +34.5% throughput gain with no measurable ROUGE regression.
- **AWQ (Activation-aware Weight Quantisation):** INT4 weight-only quantisation that calibrates per-channel scaling factors to minimise the effect on outlier activations. Requires no retraining and achieves near-lossless quality.
- **GPTQ:** INT4 post-training quantisation using second-order reconstruction. Broader model support than AWQ, slightly lower throughput on Hopper.
- **INT8 (W8A16):** weights stored as INT8, dequantised to FP16 on-the-fly during computation. Balances memory savings and compatibility across all NVIDIA GPU generations.

Prefix caching.

When the same prompt prefix appears across many requests, vLLM avoids recomputing its KV cache by retaining computed blocks in GPU memory under a hash-based lookup mechanism. Each KV cache block is identified by a hash of the token IDs it contains and its position in the layer stack. On a cache hit, the block is immediately reused via the block table with no additional computation. On a miss, the block is computed normally and inserted into the cache for future reuse.

Cache eviction follows a least-recently-used (LRU) policy: when the KV cache block pool is exhausted, the blocks accessed longest ago are freed first. On the CNN/DailyMail summarisation task, all 13 368 prompts share a common instruction template, making this feature highly effective. In vLLM 0.17.1, automatic prefix caching (APC) is enabled by default in the V1 engine; disabling it costs −3.1% throughput on this workload (see Section 9).

Chunked prefill.

A long prompt occupies the GPU entirely during its prefill phase, blocking all other decode steps. Chunked prefill breaks the prompt into fixed-size token chunks (configurable via `max_num_batched_tokens`) and interleaves these chunks with ongoing decode iterations from other requests. This bounds the worst-case time-to-first-token (TTFT) for co-located requests and keeps the GPU executing useful decode work concurrently.

The trade-off is slightly increased total latency for the chunked request itself (due to the overhead of multiple scheduling steps) in exchange for substantially better latency for all other concurrent requests. In our experiments, chunked prefill added a modest +0.8% throughput gain under vLLM 0.10.0. Under vLLM 0.17.1, where chunked prefill is enabled by default with an automatically tuned token budget, fully disabling it

(`enable_chunked_prefill=False`) caused a −14.4% throughput regression (685.88 → 587.32 tok/s), confirming that the feature is load-bearing in the V1 engine. Partially neutralising chunked prefill by constraining the token budget to one maximum-length sequence (`max_num_batched_tokens=2668`, `exp_06`) produced only a −1.8% regression, as the scheduling machinery remained active but no prompt was ever split across multiple chunks.

Async scheduling and multi-step scheduling.

The decode loop in vLLM consists of alternating *scheduling* (CPU-side) and *execution* (GPU-side) phases. In the default synchronous mode, the CPU waits for the GPU to finish the current decode step before computing the next schedule.

- **Async scheduling:** the CPU scheduler runs concurrently with the GPU executing the previous iteration's kernel, hiding scheduling latency behind GPU compute. In this work, async scheduling added a reproducible +4.9% throughput gain in the vLLM 0.10.0 feature-toggle study and +3.0% in the vLLM 0.17.1 single-variable study.
- **Multi-step scheduling:** the scheduler commits to N consecutive decode steps and dispatches them to the GPU in a single batch, bypassing per-step CPU–GPU synchronisation for those N steps. At the large batch sizes used here (512–2048), multi-step scheduling was counter-productive (−10% to −12%) because the lost request admission flexibility outweighed the synchronisation savings. This is consistent with vLLM's own guidance that multi-step scheduling is more beneficial for small-batch, latency-sensitive workloads.

Speculative decoding.

Autoregressive decoding is inherently serial: each token requires a full forward pass through the model. Speculative decoding [16] breaks this serialisation by using a small *draft model* to propose k candidate tokens in a single pass, then verifying them with the target model in parallel. At 100% acceptance rate, throughput increases by up to $k\times$; in practice, acceptance rates are below 100%, so real gains depend on the draft model quality and the similarity between draft and target distributions. This technique is more beneficial for latency-sensitive scenarios than for the Offline throughput setting evaluated here, and is discussed further in Section 18.

Attention backends.

vLLM supports multiple attention kernel backends: the default `FLASH_ATTN` (FlashAttention-2/3 [15]), `FLASHINFER` (a more recent kernel library with additional optimisations), and a fallback `XFORMERS` backend. In our experiments, switching from the default FlashAttention to FlashInfer produced a −0.1% change in throughput, effectively neutral, indicating that on Hopper both backends are comparably optimised for this workload.

Tensor parallelism and disaggregated prefill.

vLLM supports multi-GPU tensor parallelism via Megatron-style column/row splitting of weight matrices across devices, controlled by the `tensor_parallel_size`

parameter. For single-GPU deployments this parameter is fixed at 1. Disaggregated prefill, which routes prefill and decode to separate GPU pools, is an emerging architecture supported in newer vLLM versions and is outside the scope of this single-GPU study; it is discussed in Section 18.

The V1 Engine.

vLLM introduced the V1 execution engine as an alpha release in January 2025, with it becoming the default in v0.8.0 and the legacy V0 engine being fully retired in v0.11.0 [2]. The V1 engine consolidates scheduling, memory management, and CUDA graph dispatch into a unified loop with the scheduler and execution core isolated in a separate process to eliminate CPU blocking. Key V1 changes relevant to this work include: (i) automatic prefix caching enabled by default, (ii) compilation defaulting to O2, (iii) a right-sized KV cache allocator that reserves only as many blocks as needed for the actual batch rather than the theoretical maximum, and (iv) improved async scheduling integration. All single-variable experiments in this report were conducted under vLLM 0.17.1 with the V1 engine active.

4.3 Competing Runtimes

While vLLM is the mandated SUT for this MLPerf task, several competing frameworks are worth noting for context:

- **TensorRT-LLM [3]:** NVIDIA's open-source serving library (with proprietary internal attention kernels), tightly integrated with TensorRT and cuDNN. It offers the highest throughput on NVIDIA hardware through aggressive kernel fusion and ahead-of-time compilation, but requires explicit engine builds per model/GPU combination and is less flexible for research experimentation.
- **SGLang [4]:** An open-source framework focused on structured generation and multi-call LLM programs. It introduces RadixAttention, which maintains the KV cache of all requests in a radix tree (compact prefix tree) with LRU eviction, enabling automatic and efficient KV cache reuse across any requests that share prompt prefixes. SGLang has shown competitive throughput with vLLM on structured generation workloads.
- **HuggingFace TGI (Text Generation Inference):** A production-grade serving framework from HuggingFace with broad model support. TGI implements continuous batching but uses batch-level scheduling with configurable timeout windows rather than vLLM's strict iteration-level admission, trading some throughput for more predictable latency. It is simpler to deploy for standard use cases but less configurable for research-oriented parameter sweeps.
- **DeepSpeed-MII:** Microsoft's inference library, optimised for multi-GPU and multi-node deployment using DeepSpeed's ZeRO memory management. It is more relevant for very large models (>70B parameters) than for the 8B model studied here, where single-GPU serving is both feasible and optimal.

5 MLPerf Inference Framework

MLPerf Inference [5] is an industry-standard benchmark suite maintained by MLCommons, an open engineering consortium launched in December 2020 and growing out of the MLPerf benchmark initiative that began in 2018. Its founding membership included representatives from Google, NVIDIA, Intel, Meta, Microsoft, Alibaba, and over 50 other organisations from industry and academia [20]. The consortium's mission is to provide fair, reproducible, and vendor-neutral benchmarks for machine learning systems, covering both training (MLPerf Training) and inference (MLPerf Inference). Results are published approximately twice yearly in open peer-reviewed rounds (e.g. v5.0, v5.1), enabling longitudinal tracking of hardware and software progress across the industry.

MLPerf Inference benchmarks span five task domains: computer vision (image classification, object detection), natural language processing (BERT, LLM summarisation and question answering), recommendation (DLRM), speech recognition, and generative AI (text-to-image). Each domain includes one or more specific model–dataset pairs with fixed accuracy targets and quality metrics. For the LLM domain, the v5.0 round [6] includes: Llama 2-70B on OpenOrca (the established generative AI benchmark), an updated Llama 2-70B Interactive variant with stricter TTFT and TPOT latency constraints, and the newly added Llama 3.1-405B benchmark covering question answering, mathematics, and code generation tasks. This work uses the Llama 3.1-8B summarisation task on CNN/DailyMail, which was introduced in an earlier round and remains an active benchmark in the suite.

5.1 Scenarios

MLPerf defines four scenarios that model distinct real-world deployment patterns. Each scenario prescribes a different query arrival process and a different primary performance metric:

For LLM workloads, two additional latency metrics are defined beyond aggregate throughput:

- **Time to first token (TTFT):** the latency from query submission to the first generated token. This measures prefill latency and is critical for interactive user experience. The Server scenario enforces a TTFT p99 target.
- **Time per output token (TPOT):** the inter-token latency during decode. The Server scenario enforces a TPOT p99 target to ensure smooth token streaming.

This work uses the **Offline scenario** exclusively, as it directly measures peak throughput without latency constraints and is the most informative scenario for characterising the throughput impact of vLLM configuration parameters. In the Offline scenario, LoadGen delivers all 13 368 CNN/DailyMail samples to the SUT in a single batch, and the SUT returns all completions as fast as possible. The reported metric is the total number of samples completed per second.

5.2 Divisions and Categories

MLPerf Inference submissions are classified along two axes:

Table 2 MLPerf Inference scenarios and their primary metrics.

Scenario	Primary metric	Description
SingleStream	Latency (p90)	One query at a time; measures single-request latency. Models edge and mobile deployments where the user waits for one result before issuing the next.
MultiStream	Latency (p99)	Multiple independent streams processed concurrently. Models autonomous driving and multi-camera video analytics where several input streams must be processed simultaneously.
Server	Achieved QPS	Queries arrive according to a Poisson process at a target rate; the system must maintain p99 tail latency below a threshold. Models cloud serving with unpredictable user traffic.
Offline	Samples/second	All queries are presented at once; the system processes them as fast as possible with no latency constraint. Models batch processing, data pipelines, and throughput benchmarking.

Divisions control the degree of model and numerical modification allowed:

- **Closed Division:** the model architecture, weights, and preprocessing must match the reference implementation exactly. Post-training quantisation is permitted provided accuracy targets are met, but retraining and weight replacement are explicitly prohibited [5]. This ensures apples-to-apples hardware comparisons.
- **Open Division:** any model, weights, or preprocessing may be used provided the same task and dataset are evaluated. This allows demonstrating novel techniques (e.g. model distillation, pruning, or retraining-based quantisation) that do not fit Closed Division constraints.

Categories indicate the commercial availability of the hardware:

- **Available:** hardware that is commercially purchasable at submission time.
- **Preview:** hardware announced but not yet commercially available.
- **Research, Development, and Internal (RDI):** prototype or pre-production hardware.

This work follows Closed Division rules with Available category hardware (the NVIDIA H200 NVL), meaning the model weights, tokeniser, and dataset preprocessing exactly match the MLPerf reference implementation for Llama3.1-8B. FP8 quantisation is applied without retraining, in compliance with Closed Division rules.

5.3 LoadGen

The MLCommons LoadGen library [5] is the canonical query driver and timing infrastructure for all MLPerf Inference submissions. It is implemented in C++ for nanosecond-precision timestamps and provides Python bindings for SUT integration. The LoadGen–SUT interaction follows a strict callback protocol:

1. **Initialisation:** LoadGen calls `QSL::LoadSamplesToRam()` to load the evaluation dataset into RAM (untimed). The SUT registers its `IssueQuery()` callback with LoadGen.
2. **Query issuance:** the timed window begins. LoadGen calls `IssueQuery()` with a set of `QuerySample` objects, each containing an index into the evaluation dataset. In the Offline scenario, all samples are issued in a single call.
3. **Response:** the SUT processes queries asynchronously using the inference engine (vLLM in this case), tokenises outputs, and calls `QuerySampleComplete()` with responses containing the generated token IDs and first-token timestamps.
4. **Flush and validation:** LoadGen calls `FlushQueries()` to signal that no further queries will be issued, then computes all timing statistics and verifies compliance constraints (minimum run duration, minimum query count, accuracy thresholds).

The LoadGen–SUT interaction protocol above is defined in the MLCommons LoadGen specification [5].

Our SUT wraps a vLLM [2] engine and translates between LoadGen's sample-index-based query interface and vLLM's text-generation API. The SUT implementation is adapted from the MLPerf reference implementation with modifications for configuration parameter injection, MLflow experiment tracking, and FP8 quantisation support.

5.4 Compliance and Reproducibility

MLPerf enforces a rigorous compliance framework to ensure that reported results are reproducible and not the product of cherry-picked runs:

- **Minimum duration and sample count:** the Offline scenario requires at least 60 seconds of execution and a minimum of min(24,576, N_{dataset}) samples processed, where N_{dataset} is the size of the evaluation dataset. For Llama3.1-8B, the CNN/-DailyMail dataset contains 13 368 samples, which is below the 24 576 threshold, so a single full pass over the dataset satisfies the minimum query count [5]. The SingleStream and Server scenarios use a 600-second minimum duration.
- **Accuracy validation:** a separate untimed accuracy run processes the entire evaluation dataset (13 368 samples for Llama3.1-8B) and computes ROUGE-1, ROUGE-2, and ROUGE-L scores. The achieved scores must meet or exceed the published accuracy targets (see Table 7).
- **Compliance tests:** MLPerf provides mandatory compliance test scripts that verify run validity. For Llama3.1-8B, the required test is TEST06 [5], which verifies that the SUT produces deterministic outputs and that no data-dependent optimisation occurs between performance and accuracy runs.

- **Server scenario latency constraints:** for the Server scenario, Llama3.1-8B enforces TTFT ≤ 2 seconds and TPOT ≤ 100 ms at the p99 percentile [6]. A TPOT of 100 ms corresponds to approximately 480 words per minute, which exceeds typical human reading rates and ensures fluid interactive use.
- **Audit logging:** all runs produce standardised log files recording software versions, configuration parameters, random seeds, dataset revisions, and per-query timestamps.

The performance sample count for Llama3.1-8B is 13 368, as defined in the official `mlperf.conf` configuration. All experiments in this work satisfy these compliance requirements unless otherwise noted.

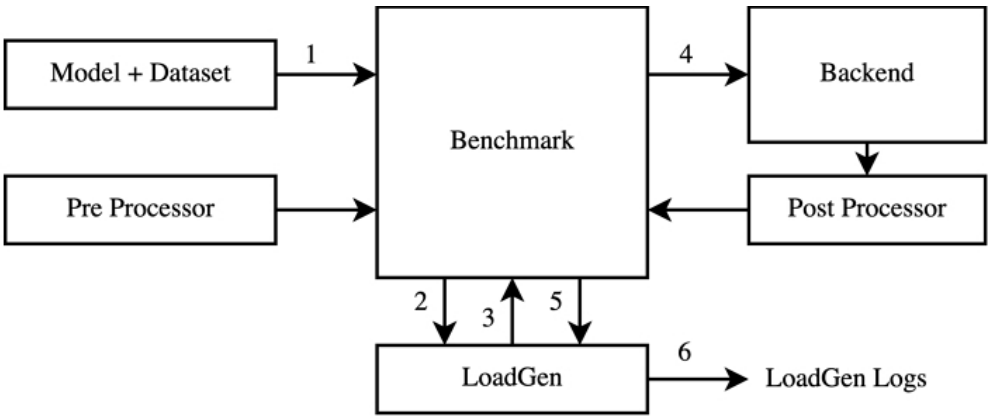

Fig. 3 LoadGen integration diagram showing the interaction between LoadGen, the System Under Test, and query/response flow. Source: MLCommons LoadGen documentation [20].

6 Transformer Architecture and Llama3.1-8B

6.1 Origins and Motivation

Before the transformer, most language models used recurrent neural networks (RNNs), including LSTMs and GRUs. These models read text one token at a time, passing a hidden state from each step to the next. This design has two key problems. First, it is inherently sequential: token t cannot be processed until token $t-1$ is complete, which means the model cannot exploit parallel hardware even for long sequences. Second, gradients must travel backwards through every step of the sequence during training, which causes them to vanish or explode, making it hard for the model to learn relationships between words that are far apart.

The *attention mechanism* was introduced by Bahdanau et al. (2014) to address the long-range dependency problem in sequence-to-sequence translation. Instead of relying on a single compressed context vector produced by the encoder, attention lets the decoder directly look up any position in the encoder output at each generation step.

This greatly improved translation of long sentences. Luong et al. (2015) later simplified and extended this with multiplicative (dot-product) attention scoring. However, in both cases the encoder itself was still a slow, sequential RNN.

Vaswani et al. [31] went further and removed recurrence completely. The *transformer* uses attention as its only mechanism for connecting different positions in a sequence. Any two tokens can exchange information in a single layer rather than having to pass information step-by-step through $O(N)$ recurrent transitions. Because each token's computation is independent of the others, the entire sequence can be processed in parallel. The transformer achieved better translation quality and trained significantly faster than RNN-based models, and it became the foundation for every major language model since.

6.2 Input Representation: Tokenisation, Embeddings, and Positional Encoding

Before text can be fed into a transformer, it needs to be turned into a sequence of fixed-size vectors. This happens in three steps.

Tokenisation.

Text is first broken into small sub-word pieces called tokens, using an algorithm such as Byte Pair Encoding (BPE) or WordPiece. This strikes a balance between two extremes: splitting into individual characters (short vocabulary, very long sequences) or keeping full words (huge vocabulary, poor handling of rare or misspelt words). Each token is assigned an integer index from a vocabulary $\mathcal{V}$.

Token Embeddings.

Each integer index is looked up in a learned embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$, converting it into a dense vector of size d_{model}:

$$\mathbf{e}_t = \mathbf{E}[x_t] \in \mathbb{R}^{d_{\text{model}}} \tag{6}$$

A sequence of N tokens therefore becomes a matrix $\mathbf{E}_{\text{tok}} \in \mathbb{R}^{N \times d_{\text{model}}}$. In the original transformer, this same embedding matrix is reused (transposed) at the output layer, which halves the number of parameters needed for the vocabulary tables.

Positional Encoding.

Self-attention treats all tokens symmetrically: shuffling the input rows produces the same attention scores in the same permuted order, so the model has no built-in sense of word order. To fix this, position information is added to each token vector. The original transformer uses fixed sinusoidal functions of position:

$$\text{PE}(\text{pos}, 2i) = \sin\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big), \qquad \text{PE}(\text{pos}, 2i+1) = \cos\Big(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\Big) \tag{7}$$

Each dimension oscillates at a different frequency so every position gets a unique pattern of values. The positional encoding is added to the token embedding before the

first transformer layer:

$$\mathbf{X}_0 = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{N \times d_{\text{model}}} \tag{8}$$

Modern models like Llama3.1 replace this with Rotary Positional Embeddings (RoPE), which encode position directly inside the attention computation and handle longer contexts more effectively (see Section 6.10).

6.3 Scaled Dot-Product Attention

Attention is the core building block of the transformer [31]. The idea is simple: for each token in the sequence, compute a weighted average of all other tokens' information, where the weights reflect how relevant each token is.

To do this, the input matrix $X \in \mathbb{R}^{N \times d_{\text{model}}}$ is projected into three separate matrices using learned weight matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \tag{9}$$

These are called the *queries* (Q), *keys* (K), and *values* (V). Think of it like a search engine: queries are what you are looking for, keys are what each token offers as a match, and values are the actual content you retrieve. Each token's query is compared against all keys via a dot product, producing a score that says how much attention to pay to each position. The scores are scaled, optionally masked, normalised with softmax, and used to take a weighted sum of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \tag{10}$$

where M is an optional mask matrix. In decoder-only models, M is a *causal mask*: setting $M_{ij} = -\infty$ for all $j > i$ means token i can only attend to tokens at positions i or earlier, preventing the model from seeing future tokens during training. Positions where $M_{ij} = 0$ are unaffected.

Why divide by $\sqrt{d_k}$?

As the dimension d_k grows, dot products between queries and keys tend to grow larger in magnitude. Very large values push the softmax towards near-zero gradients, where nearly all the attention weight collapses onto the single best-matching key. Dividing by $\sqrt{d_k}$ keeps the scores in a range where softmax behaves well during training.

Computational cost.

Computing the full $N \times N$ score matrix requires $O(N^2 d_k)$ operations per attention head per layer. This quadratic scaling in sequence length N is the main bottleneck for long contexts: doubling the sequence length quadruples the attention cost. FlashAttention [15] reduces the memory footprint by tiling the computation and avoiding materialising the full $N \times N$ matrix in HBM, but the total arithmetic work remains $O(N^2)$.

6.4 Multi-Head Attention

A single attention head produces one weighted mixture of all token representations [31]. *Multi-head attention* (MHA) runs h such heads in parallel, each with its own projection matrices. This lets different heads focus on different kinds of relationships simultaneously: one head might track syntactic structure while another tracks coreference. The outputs of all heads are concatenated and linearly projected:

$$\begin{aligned} \mathrm{MHA}(X) &= \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\, W^O \\ \mathrm{head}_i &= \mathrm{Attention}(XW_i^Q,\, XW_i^K,\, XW_i^V) \end{aligned} \tag{11}$$

where $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are each head's projection matrices, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ mixes the concatenated head outputs back into d_{model} dimensions.

With standard MHA and $d_k = d_v = d_{\text{model}}/h$, the four projection matrices (W^Q, W^K, W^V, W^O) together contribute $4d_{\text{model}}^2$ parameters per layer. However, Llama3.1-8B uses Grouped Query Attention (GQA) with 8 KV heads rather than 32, so W^K and W^V are each $d_{\text{model}} \times d_{\text{kv}}$ where $d_{\text{kv}} = 8 \times 128 = 1{,}024$, reducing total attention parameters from the naïve 2.1B to approximately 1.34B across all 32 layers. The FFN weights account for the remaining majority of the parameter budget, as discussed in Section 6.10.

6.5 Position-Wise Feed-Forward Sub-Layer

After the attention sub-layer, each transformer layer passes every token through a small feed-forward network (FFN) independently [31]. The same FFN is applied to every position, but each position is processed on its own with no interaction between tokens. In the original transformer, this is a two-layer network with a ReLU activation:

$$\mathrm{FFN}(x) = \max(0,\, xW_1 + b_1)\, W_2 + b_2 \tag{12}$$

where W_1 expands the representation from d_{model} to a larger hidden size $d_{\text{ff}} = 4d_{\text{model}}$, and W_2 projects it back down. The expansion gives the model a wider space to apply non-linear transformations before the residual connection adds the result back.

The two weight matrices contribute approximately $2\, d_{\text{model}}\, d_{\text{ff}} \approx 8\, d_{\text{model}}^2$ parameters, twice the standard MHA count, so the FFN dominates each layer. Modern architectures such as Llama3.1-8B replace ReLU with SwiGLU (Section 6.10), which adds a third weight matrix (W_{gate}, W_{up}, W_{down}). Rather than using $d_{\text{ff}} = 4d_{\text{model}}$, Llama3.1-8B uses $d_{\text{ff}} = 14{,}336$ (a ratio of 3.5× rather than 4×), chosen so that the three-matrix SwiGLU FFN is compute-equivalent to the two-matrix ReLU FFN. The effective parameter count

6.6 The Encoder Stack

The encoder reads the entire input sequence and turns it into a rich set of vectors, one per token, that capture meaning in context [31]. It is a stack of N_e identical

layers (6 in the original transformer, 24 in BERT-Large). Each layer takes the previous layer's output and refines it through two sub-layers in sequence: first a multi-head self-attention block, then a feed-forward network. Using pre-norm, this looks like:

$$\tilde{\mathbf{H}}^{(\ell)} = \mathbf{H}^{(\ell-1)} + \mathrm{MHA}\Big(\mathrm{Norm}(\mathbf{H}^{(\ell-1)})\Big) \tag{13}$$

$$\mathbf{H}^{(\ell)} = \tilde{\mathbf{H}}^{(\ell)} + \mathrm{FFN}\Big(\mathrm{Norm}(\tilde{\mathbf{H}}^{(\ell)})\Big) \tag{14}$$

The key property of the encoder is that its attention is fully *bidirectional*: every token can attend to every other token in the sequence with no masking. This means by the time a token reaches the final layer, its vector has been influenced by every word to its left and right simultaneously. A word like "bank" ends up with a completely different vector depending on whether the surrounding words are about money or rivers.

After N_e layers, the final encoder output $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{N \times d_{\text{model}}}$ contains one contextualised vector per input token. These vectors are what the decoder reads from when generating output.

6.7 The Decoder Stack

The decoder generates output one token at a time, using both what has been generated so far and the encoder's understanding of the input [31]. Each of its N_d layers has *three* sub-layers instead of two: masked self-attention, cross-attention, and a feed-forward network.

Sub-layer 1: Masked self-attention.

The decoder attends to its own previously generated tokens. To prevent it from cheating during training by reading ahead, its self-attention uses a causal mask so that position t can only see positions $1, \ldots, t$:

$$\tilde{\mathbf{Y}}^{(\ell)} = \mathbf{Y}^{(\ell-1)} + \mathrm{MHA}_{\text{causal}}\Big(\mathrm{Norm}(\mathbf{Y}^{(\ell-1)})\Big) \tag{15}$$

Without this mask, the model could trivially look up the correct answer from future positions during teacher-forced training and would never learn to generate properly.

Sub-layer 2: Cross-attention.

This is where the decoder reads the source sequence. The queries come from the decoder's current hidden states, telling the attention mechanism "what am I trying to say right now?". The keys and values come from the encoder output, providing the source content to read from:

$$Q = \tilde{\mathbf{Y}}^{(\ell)} W^Q, \qquad K = \mathbf{H}_{\text{enc}}\, W^K, \qquad V = \mathbf{H}_{\text{enc}}\, W^V \tag{16}$$

$$\hat{\mathbf{Y}}^{(\ell)} = \tilde{\mathbf{Y}}^{(\ell)} + \mathrm{Attention}(Q, K, V) \tag{17}$$

There is no causal mask here: each decoder position can attend freely to any encoder position. At each generation step the cross-attention head selects which source tokens

are most relevant to what is being generated, which is how the model decides which part of an article to focus on when summarising.

Sub-layer 3: Position-wise FFN.

Identical in form to the encoder FFN, applied after cross-attention:

$$\mathbf{Y}^{(\ell)} = \hat{\mathbf{Y}}^{(\ell)} + \mathrm{FFN}\Big(\mathrm{Norm}(\hat{\mathbf{Y}}^{(\ell)})\Big) \tag{18}$$

Output projection and next-token probability.

After the final decoder layer, the hidden state at the last generated position is projected back into vocabulary space via a linear layer. This layer shares weights with the token embedding matrix (weight tying), which halves the vocabulary parameter count. A softmax then converts the raw scores (logits) into a probability distribution over the next token:

$$P(x_{T+1} \mid x_{\leq T}) = \mathrm{softmax}\Big(\mathbf{Y}_T^{(N_d)} \cdot \mathbf{E}^T\Big) \in \mathbb{R}^{|\mathcal{V}|} \tag{19}$$

Under greedy decoding the highest-probability token is always chosen. Under temperature sampling, the logits are divided by a temperature τ before softmax: high τ flattens the distribution (more random output), low τ sharpens it (more predictable output).

6.8 The Complete Encoder-Decoder Model

Putting it all together [31]: the encoder reads the source sequence once and produces a fixed set of contextualised vectors. The decoder then generates the output sequence one token at a time, at each step attending to both its own previous outputs and the encoder's vectors.

During training, the decoder does not wait for its own predictions. Instead, the ground-truth target sequence is fed in shifted one position to the right (a technique called *teacher forcing*), so all T output positions can be processed in a single parallel forward pass. The model is trained to minimise the total cross-entropy loss across all target positions:

$$\mathcal{L}_{\mathrm{seq2seq}} = -\sum_{t=1}^{T} \log P(y_t \mid y_1, \ldots, y_{t-1},\, x_1, \ldots, x_N;\, \theta) \tag{20}$$

At inference time, teacher forcing is unavailable: the model generates tokens one at a time and feeds each output back in as the next decoder input. This gap between training conditions (ground-truth inputs) and inference conditions (model's own potentially wrong outputs) is called *exposure bias*. In practice it is managed through large-scale pre-training and instruction tuning, which teach the model to recover gracefully from its own errors.

6.9 Architectural Variants: Encoder-Only, Decoder-Only, Encoder-Decoder

The original transformer uses both an encoder and a decoder. Later work found that dropping one half produces simpler, more specialised models suited to different tasks. The three resulting variants are summarised in Table 3.

Table 3 The three main transformer architectural variants.

Variant	Representative models	Primary use case
Encoder-only	BERT, RoBERTa, DeBERTa	Classification; NER; extractive QA
Decoder-only	GPT-2/3, Llama, Mistral, DeepSeek	Text generation; chat; coding
Encoder-decoder	T5, BART, mT5, original Transformer	Machine translation; summarisation; seq2seq

Encoder-only models.

Models like BERT [32] and RoBERTa keep only the encoder. Because every token sees the full sequence in both directions, these models build up very rich contextual representations, well suited for tasks like classification, named entity recognition, and reading comprehension. However, they cannot generate text, as there is no mechanism to produce tokens one at a time autoregressively.

Decoder-only models.

Models like GPT-2, GPT-3 [9], and all Llama variants [11–13] keep only the decoder. A causal mask ensures each token only attends to previous tokens, making the model naturally suited for left-to-right generation tasks: chat, code completion, and instruction following. At inference time, the entire context (prompt plus generated tokens) lives in the KV cache, making KV cache management the central engineering challenge, which is exactly what PagedAttention [2] addresses (Section 4.2).

Encoder-decoder models.

Models like T5 [33] and BART keep both halves. The encoder reads the full source bidirectionally, and the decoder generates the target using cross-attention over those encoder representations. This setup works best for tasks with a clear source-to-target structure such as translation or summarisation. The main downside for serving is memory: both encoder and decoder weights must be resident simultaneously.
Llama3.1-8B is a **decoder-only** model. For the CNN/DailyMail summarisation task, the source article and a fixed instruction template are concatenated to form the prompt, and the model generates the summary autoregressively. No separate encoder

exists; the entire prompt-plus-summary context is managed in the decoder's KV cache throughout generation.

6.10 Llama3.1-8B Architecture

Llama3.1-8B [11] is a decoder-only causal language model in Meta's Llama 3.1 family. It was released in July 2024 and trained on more than 15 trillion tokens of text drawn from a diverse multilingual corpus, with English as the dominant language and additional support for German, French, Italian, Portuguese, Hindi, Spanish, and Thai [**?** **?**]. Training used large GPU clusters with 4D parallelism (tensor, pipeline, context, and data parallelism) to scale efficiently across thousands of NVIDIA H100 GPUs [11].

Within the family, the 8B-parameter model is designed as a practical sweet spot: small enough to serve on a single high-memory GPU while still achieving strong benchmark performance. On MMLU, Llama3.1-8B reaches a score of 73.0 in a 0-shot chain-of-thought setting, compared to 86.0 for the 70B model and 88.6 for the 405B model [**?**]. This places the 8B model in the same rough performance tier as much larger previous-generation models, showing the benefit of scaling training data to around 15T tokens.

Table 4 summarises the key architecture parameters.

Table 4 Llama3.1-8B model architecture parameters.

Parameter	Value
Architecture	`LlamaForCausalLM` (decoder-only)
Hidden size (d_{model})	4 096
Number of layers	32
Attention heads (query)	32
Key-value heads (GQA)	8 (groups of 4 query heads)
FFN intermediate size	14 336
Vocabulary size	128 000 base tokens + 256 special tokens [**?**]
Max context length	131 072 tokens (128K)
Activation function	SwiGLU (built from SiLU)
Positional encoding	RoPE with high base θ and 8.0x scaling for 128K context [**?**]
Norm type	RMSNorm ($\varepsilon = 10^{-5}$)
Native precision	BFloat16
Training token count	15T+ tokens (multilingual)

Grouped-Query Attention (GQA).

Grouped-Query Attention (GQA) [27] sits between two extremes: standard multi-head attention, where each query head has its own key and value projections ($h_{kv} = h$), and multi-query attention, where all heads share a single key and value projection ($h_{kv} = 1$). In GQA, keys and values are shared within small groups of query heads.

For Llama3.1-8B the configuration is $h = 32$ query heads and $h_{kv} = 8$ key–value heads, giving groups of size $g = h/h_{kv} = 4$. For group j and head index i inside that group, the per-head computation is:

$$\text{head}_{j,i} = \text{Attention}\big(XW^Q_{j,i},\ XW^K_j,\ XW^V_j\big), \qquad i = 1, \ldots, g \tag{21}$$

Each query head has its own $W^Q_{j,i}$, while W^K_j and W^V_j are shared across the group. Sharing the key and value projections cuts both the number of parameters in W^K and W^V and the per-token KV cache size by a factor of g compared to fully independent KV heads [27]. For Llama3.1-8B this reduces the KV cache from the 32-head baseline to the 8-head GQA configuration, giving a 4× reduction in key/value storage per token.

The exact per-token KV cache size in BF16 can be written as:

$$\text{KV bytes/token} = L \cdot h_{kv} \cdot d_k \cdot 2 \cdot \text{sizeof(BF16)} \tag{22}$$

where $L = 32$ layers, $h_{kv} = 8$ heads, $d_k = 128$ per-head dimension, and sizeof(BF16) = 2 bytes. Plugging in these values yields $32 \cdot 8 \cdot 128 \cdot 2 \cdot 2 = 131\,072$ bytes ($\approx 128\,\text{KB}$) per token, which is the figure used in our KV cache capacity calculations.

Rotary Positional Embeddings (RoPE).

Rotary Positional Embeddings (RoPE) [28] encode position by rotating each pair of coordinates in the query and key vectors by an angle proportional to the token index. For a token at position m, each consecutive pair of dimensions $(2i, 2i+1)$ is rotated by angle $m\theta_i$:

$$\text{RoPE}(x, m)_{2i} = x_{2i}\cos(m\theta_i) - x_{2i+1}\sin(m\theta_i), \tag{23}$$

$$\text{RoPE}(x, m)_{2i+1} = x_{2i}\sin(m\theta_i) + x_{2i+1}\cos(m\theta_i) \tag{24}$$

with $\theta_i = \theta_{\text{base}}^{-2i/d_k}$. Llama3.1 uses a much larger base angle (on the order of 5×10^5) together with a RoPE scaling factor of about 8.0 to extend the effective context window from its original 8 192-token training window to 128 K tokens without retraining the core weights [**?**]. A key property of RoPE is that the inner product between a rotated query at position m and a rotated key at position n depends only on the relative offset $(m - n)$, giving the model an implicit sense of relative distance.

SiLU activation and SwiGLU.

The FFN in each Llama layer uses a SwiGLU block [29], a gated variant of the standard two-layer ReLU FFN. Instead of applying a single non-linearity, SwiGLU uses two

parallel linear transforms and a SiLU gate:

$$\mathrm{FFN}(x) = \big(\mathrm{SiLU}(xW_{\mathrm{gate}}) \odot xW_{\mathrm{up}}\big)W_{\mathrm{down}} \tag{25}$$

where $\mathrm{SiLU}(z) = z \cdot \sigma(z)$, σ is the sigmoid function, and $\odot$ denotes element-wise multiplication. The matrices $W_{\mathrm{gate}}, W_{\mathrm{up}} \in \mathbb{R}^{d_{\mathrm{model}} \times d_{\mathrm{ff}}}$ and $W_{\mathrm{down}} \in \mathbb{R}^{d_{\mathrm{ff}} \times d_{\mathrm{model}}}$. This gating lets the model decide, dimension by dimension, how much of the expanded representation to keep. Empirically, SwiGLU improves language modelling perplexity at the same compute budget compared to a ReLU FFN [29].

RMSNorm.

RMSNorm [30] replaces LayerNorm by removing the mean subtraction step and normalising only by the root mean square of the activations:

$$\mathrm{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}} \odot \gamma \tag{26}$$

where $\gamma \in \mathbb{R}^d$ is a learned scale, d is the feature dimension, and $\varepsilon \approx 10^{-5}$ prevents division by zero. There is no learned bias. Dropping the mean-reduction and bias-addition makes each normalisation call slightly cheaper, which adds up over dozens of norms per forward pass and many decode steps. In practice, RMSNorm provides similar training stability to LayerNorm at lower cost, which is why it is used in all Llama versions.

Tokeniser.

Llama3.1 uses a Tiktoken-style BPE tokeniser with a base vocabulary of 128 000 tokens plus 256 reserved special tokens [**?**]. Compared to the 32 000-token vocabularies used in Llama 1 and 2, the larger vocabulary allows the tokeniser to represent common words and multi-character chunks more compactly, especially in non-English languages. For English text, this reduces the number of tokens needed for a given passage, which in turn slightly lowers prefill compute and KV cache usage for fixed-length documents.

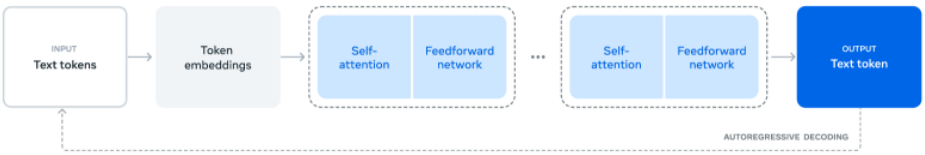

Fig. 4 High-level Llama 3/3.1 architecture and training setup (adapted from Meta's Llama 3 technical report [11]).

7 Experimental Setup

7.1 Hardware

All experiments ran on a single node with one NVIDIA H200 NVL GPU. Table 5 lists the hardware.

Table 5 Hardware configuration.

Component	Specification	Notes
GPU	NVIDIA H200 NVL, 141 GB HBM3e	Compute capability 9.0
GPU count	1	PCIe attached
GPU clocks	1 785 MHz (core) / 3 201 MHz (mem)	Max boost
GPU TDP	700 W	H200 NVL rated TDP
CPU	2× Intel Xeon Platinum 8480+	56 cores / 112 threads each
Total threads	224	Hyper-threading enabled
CPU frequency	2 000–3 800 MHz	Base / turbo boost
System memory	251 GB DDR5	2 NUMA nodes
Storage (OS)	894 GB SAS SSD	
Storage (data)	4× 7 TB NVMe	Model and dataset
OS kernel	4.18.0-477 (RHEL 8)	x86_64
NVIDIA driver	570.133.20	

7.2 Software Stack

The software environment was managed using a Python virtual environment. The environment was based on Python 3.12.13, with all key packages pinned for reproducibility. Table 6 lists the key components.

7.3 CNN/DailyMail Summarisation Dataset

The benchmark uses the CNN/DailyMail abstractive summarisation dataset, originally introduced by Hermann et al. [18] and further developed by See et al. [35] as a standard test bed for sequence-to-sequence models. The dataset pairs news articles from CNN and Daily Mail archives with human-written bullet-point highlights, which serve as reference summaries of 3–5 sentences.

Dataset statistics.

CNN/DailyMail v3.0.0 contains 312,971 article–summary pairs across three splits: 287,113 for training, 13,368 for validation, and 11,490 for test [35]. The MLPerf evaluation slice is the *validation split* exactly, comprising all 13,368 samples, which ensures

Table 6 Key software versions.

Component	Version	Notes
Python	3.12.13	Virtual environment
vLLM	0.17.1	PagedAttention, continuous batching, V1 engine
PyTorch	2.10.0+cu129	CUDA 12.9
Transformers	4.57.6	Pinned
Tokenizers	0.22.2	HuggingFace fast tokeniser
FlashInfer	0.6.4	Alternative attention backend
NumPy	2.2.6	
CUDA Python	12.9.4	
cuPy	14.0.1	
HuggingFace Hub	0.36.2	Model caching
MLPerf LoadGen	6.0.9	Query generation and timing
MLflow	3.10.1	Experiment tracking

direct comparability across all Llama3.1-8B submissions globally. Articles average around 781 tokens and reference summaries around 56 tokens, yielding a compression ratio of approximately 14×. The task requires the model to identify the most salient information and rephrase it concisely, making it a genuine test of *abstractive* summarisation rather than extraction.

Why CNN/DailyMail for MLPerf?

MLCommons selected CNN/DailyMail for the Llama3.1-8B task for several reasons:

- It is publicly available and well-studied, enabling independent reproducibility.
- Input lengths (up to ≈2 540 tokens) exercise the model's KV cache management without approaching the 128K context limit, keeping runtimes tractable.
- Evaluation via ROUGE scores [19] is automated and deterministic, with published BF16 reference targets that establish a clear pass/fail threshold.
- Greedy decoding produces deterministic outputs, enabling exact run-to-run reproducibility regardless of submission platform.

ROUGE evaluation metrics.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [19] measures n-gram overlap between generated and reference summaries. The four variants used are:

- **ROUGE-1:** unigram overlap (individual word recall and precision).
- **ROUGE-2:** bigram overlap (consecutive word pairs).
- **ROUGE-L:** longest common subsequence F-measure; captures sentence-level fluency and order.
- **ROUGE-Lsum:** ROUGE-L computed sentence-by-sentence across the summary; better reflects multi-sentence structure.

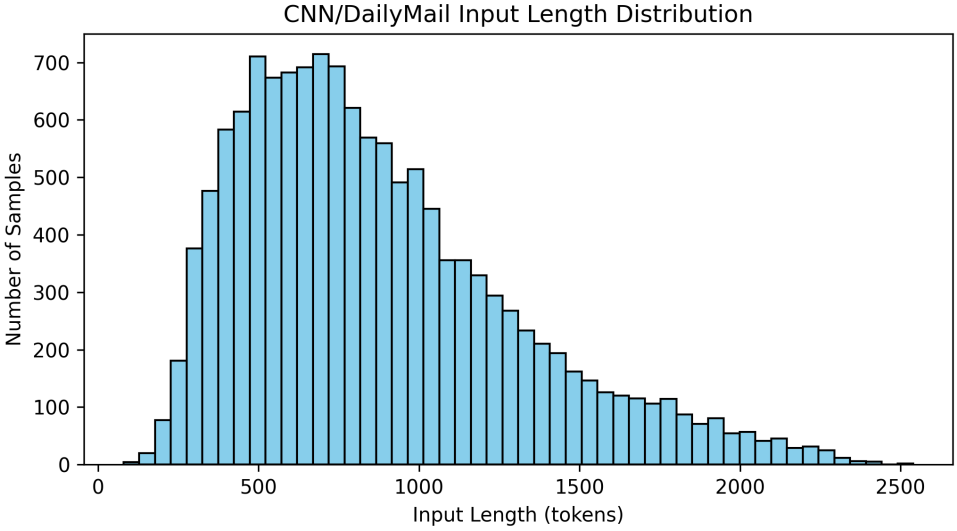

Fig. 5 Distribution of tokenised input lengths in the CNN/DailyMail MLPerf evaluation slice. The long tail up to ≈2 540 tokens motivates the `max_model_len = 2668` optimisation.

All scores are F-measures on a 0-100 scale, computed via the HuggingFace `evaluate` [21] library with NLTK word tokenisation.

Pre-processing and MLPerf format.

The dataset is delivered pre-tokenised as `cnn_eval.json`, containing `input_ids` arrays, `input_lens`, and `targets` (reference strings). Using pre-tokenised inputs ensures all submissions process identical token sequences and eliminates tokeniser-version variability as a confound. The key numerical parameters are:

- **Total samples:** 13 368 (as mandated by `mlperf.conf`).
- **Max input length:** ≈ 2 540 tokens.
- **Max output tokens:** 128 (baseline) or 64/256 (experiments).
- **Effective max sequence length:** $2\,540 + 128 = 2\,668$ tokens.

7.4 Accuracy Targets

The BF16 reference accuracy targets provided by MLCommons are shown in Table 7. The compliance threshold is 99% of the BF16 reference for all ROUGE metrics and 90% for `gen_len`.

Accuracy is evaluated in a dedicated run under the same Offline scenario used for performance measurement; both performance and accuracy runs must pass for a submission to be considered compliant [5].

Table 7 MLPerf Llama3.1-8B accuracy targets and compliance thresholds.

Metric	Target	Threshold (%)	Min allowed	Rule
ROUGE-1	38.7792	99	38.3914	$\geq$ 99% of target
ROUGE-2	15.9075	99	15.7484	$\geq$ 99% of target
ROUGE-L	24.4957	99	24.2507	$\geq$ 99% of target
ROUGE-Lsum	35.7930	99	35.4351	$\geq$ 99% of target
gen_len	8 167 644	90	7 350 880	$\geq$ 90% of target
gen_num	13 368	—	13 368	exact

8 Methodology

8.1 Serving Stack

The SUT wraps a vLLM `LLM` engine. vLLM implements *PagedAttention* [2], which manages the KV cache as virtual-memory-style pages, reducing fragmentation and enabling larger effective batch sizes. The engine supports *continuous batching*, where new requests join a running batch as earlier ones complete, keeping the GPU busy throughout.

Greedy decoding is used for determinism: `temperature=0.0`, `top_p=1`, `top_k=1`, `seed=42`. Output token IDs are converted to `np.int32` arrays before being returned to LoadGen.

8.2 Experiment Phases

Phase 1: Single-variable study.

Starting from the vLLM 0.17.1 V1 BF16 baseline (batch size 16, `max_model_len=131072`, all V1 defaults active), each vLLM parameter was changed individually with a single run per configuration. Nineteen configurations were tested across six categories: V1 default features (automatic prefix caching, async output processing), compilation levels (O0–O3), batched token budgets, scheduling (async scheduling, sort by input length), quantisation (FP8 weights, FP8 KV cache, combined FP8), batch sizes (1024, 2048), and memory and attention backend options.

Phase 2: Feature-toggle study.

Building on prior work, this phase started from an optimised baseline (FP8, `tensor_parallel_size=1`, `batch_size=1024`, `max_model_len=2668`, sorting, expandable segments) under vLLM 0.10.0 and toggled features one at a time. Group A covered engine features (chunked prefill, batched token budget, GPU memory utilisation 0.98, prefix caching, scheduler delay). Group B covered scheduling and compilation (async scheduling, multi-step scheduling, `torch.compile`, FlashInfer). Group C tested the most promising combinations.

Phase 3: Accuracy study.

A sweep of 22 configurations was tested in accuracy mode, covering sampling parameters, output token budgets, batch sizes, precision variants, context length limits, and engine features. All runs used BF16 precision and greedy decoding unless otherwise stated.

8.3 Baseline vLLM Engine Configuration

The baseline configuration used for throughput experiments:

Listing 1 Baseline vLLM engine configuration.

```
LLM(
    dtype                     = "auto",
    quantization              = None,
    tensor_parallel_size      = 1,         # single GPU deployment
    gpu_memory_utilization    = 0.90,
    max_model_len             = 131072,      # Default on V1
    max_num_seqs              = 256,
    enable_prefix_caching     = True,
    enable_chunked_prefill    = True,
    enforce_eager             = False,
    cpu_offload_gb            = 0,
    scheduler_delay_factor    = 0.0,
)
```

8.4 LoadGen Configuration

From `user.conf`:

- Offline: `min_duration = 600 000` ms (600 seconds; stricter than the 60-second compliance minimum, chosen to ensure stable throughput measurement).
- Server: `target_qps = 0.5`, `min_duration = 120 000` ms.
- TTFT bound: 2 000 ms; TPOT bound: 100 ms (Server scenario only).

9 Throughput Results: Single-Variable Sweep

This study measured the individual contribution of each parameter to offline throughput under vLLM 0.17.1's V1 engine. Unlike the feature-toggle study (Section 12), which started from a minimal BF16 configuration and added features one by one, the design here is inverted: the *baseline* is the stock V1 default configuration (BS=16, BF16 precision, `max_model_len=131 072`), and each experiment changes exactly one parameter. This reveals not only the gain from enabling a feature but also the cost of *removing* features that V1 already enables by default, such as automatic prefix caching (APC) and asynchronous output processing.

The baseline achieved **1 082.82 tok/s**. All runs were VALID and exhibited zero run-to-run variance at single-sample precision, confirming fully deterministic behaviour under greedy decoding.

Key findings.

FP8 quantisation (exp_10) remains the most cost-effective single optimisation at the baseline batch size, delivering a +34.5% throughput gain (1 083 → 1 456 tok/s) with no accuracy regression. vLLM's `quantization=fp8` applies W8A8 quantisation (weights and activations), exploiting Hopper's native FP8 GEMM instructions for a reliable and durable advantage. The magnitude of this gain confirms that Hopper's native FP8 GEMM instructions provide a substantial and reliable throughput advantage for memory-bound inference workloads. Enabling **FP8 KV cache alone (exp_11)** adds a further +4.5% on top of the BF16 baseline, confirming that memory-bandwidth reduction from a smaller KV cache is a measurable independent gain. However, **combining FP8 weights with FP8 KV cache (exp_12)** yielded only +18.0%, substantially less than FP8 weights alone. This subadditive behaviour likely reflects that weight and KV cache quantisation target different memory bottlenecks and do not combine linearly, a regime worth investigating further.

Batch size scaling dominates throughput once KV memory is made available. Moving to BS=1024 (exp_13) more than doubles throughput (+112.7%), and BS=2048 (exp_14) adds only a further 7.1 percentage points over BS=1024, suggesting the GPU is near saturation around BS=1024 at the baseline `max_model_len`. This diminishing marginal return suggests the GPU compute capacity is the binding constraint beyond BS=1024, and further batch size increases yield negligible benefit.

Right-sizing the KV cache (exp_00b), by setting `max_model_len=2668` to match the CNN/DailyMail dataset's 2 540-token ceiling, yields a +7.6% gain at BS=16 without any other change. This reflects V1's aggressive default KV cache pre-allocation: reserving memory for sequences far longer than the dataset requires wastes allocator capacity, and eliminating that waste yields a measurable gain even before batch sizes are scaled up..

V1 default features carry measurable value. Disabling **automatic prefix caching** (exp_01, −3.1%) and **async output processing** (exp_02, −7.0%) both degrade performance, confirming that V1's enabled-by-default choices are well-chosen for this workload. The −7.0% penalty from disabling asynchronous output processing is particularly notable, reflecting the cost of serialising token detokenisation with the GPU execution loop.

Table 8 Single-variable study results on vLLM 0.17.1 V1 engine. Each row changes only the stated parameter from the stock V1 BF16 baseline (BS=16, `max_model_len=131072`). Speedup is relative to exp_00.

ID	Description	Tok/s	Speedup
exp_00	BASELINE: stock V1 defaults, BS=16	1 082.82	—
exp_00b	Right-sized KV: `max_model_len=2668`	1 165.24	+7.6%
V1 default features (cost of disabling)			
exp_01	Disable automatic prefix caching (APC)	1 048.73	−3.1%
exp_02	Disable async output processing	1 006.82	−7.0%
Compilation level			
exp_03	`compilation_config=O1` (below V1 default O2)	1 062.58	−1.9%
exp_04	`compilation_config=O0` (no compile)	857.43	−20.8%
exp_05	`compilation_config=O3` (above V1 default O2)	1 098.13	+1.4%
Chunked prefill and batched token budget			
exp_06	`max_num_batched_tokens=2668` (neutralise chunked prefill)	1 063.47	−1.8%
exp_08	`max_num_batched_tokens=8192`	1 120.72	+3.5%
exp_09	`max_num_batched_tokens=65536`	1 089.07	+0.6%
Scheduling			
exp_07	`async_scheduling=True` (experimental)	1 115.30	+3.0%
exp_15	Sort queries by input length	1 115.30	+3.0%
Quantisation			
exp_10	`quantization=fp8` (W8A8: weights + activations)	1 456.36	**+34.5%**
exp_11	`kv_cache_dtype=fp8` (KV cache only)	1 131.84	+4.5%
exp_12	FP8 weights + FP8 KV cache	1 277.73	+18.0%
Batch size			
exp_13	`max_num_seqs=1024`	2 302.79	**+112.7%**
exp_14	`max_num_seqs=2048`	2 379.46	+119.8%
Memory and attention backend			
exp_16	`gpu_memory_utilization=0.95`	1 104.48	+2.0%
exp_17	`attention_backend=FLASHINFER`	1 081.93	−0.1%
exp_18	`skip_tokenizer_init=True`	1 083.77	+0.1%

Compilation level matters significantly. The V1 engine defaults to O2. Moving to O0 (no compilation, exp_04) causes a −20.8% regression, the largest single negative result in the sweep, while O1 (exp_03) costs −1.9%. Conversely, O3 (exp_05) gains +1.4% over the O2 default, suggesting that piecewise compilation offers a small incremental benefit beyond the default level. The O0 result underscores that CUDA graph capture and `torch.compile` are not optional micro-optimisations in V1: they are load-bearing components of the engine's performance model.

Batched token budget tuning shows a clear sweet spot. Capping `max_num_batched_tokens=2668` effectively disables chunked prefill by constraining the budget to one maximum-length sequence (exp_06), at a cost of −1.8%. Raising it to 8 192 (exp_08) adds +3.5%, the best result among batched-token variants, while 65 536

(exp_09) adds only +0.6% as the extra budget provides no additional benefit once the active batch fits within 8K tokens.

Async scheduling (exp_07) and **sort by input length (exp_15)** each add exactly +3.0%. Both operate orthogonally (scheduling pipeline overlap versus padding waste reduction), so their gains are likely additive when combined.

FlashInfer (exp_17) is now effectively neutral (−0.1%). This is likely attributable to matured FlashInfer integration in the V1 engine and better alignment with the H200's memory layout. It is no longer a reason to avoid the backend, though it offers no advantage over the default either.

GPU memory utilisation at 0.95 (exp_16) provides a modest +2.0% gain. The fact that this setting is safe here, whereas it would cause out-of-memory failures at the full **max_model_len=131072**, confirms that right-sizing the context window (exp_00b) is a prerequisite for safely raising memory utilisation.

Skipping tokeniser initialisation (exp_18) has a negligible effect (+0.1%), as expected: tokenisation overhead is fully amortised across the 600-second offline run window.

Figure 6 visualises the single-variable sweep reported above. For clarity, Figure 7 isolates the joint effect of precision and batch size, and Figure 8 summarises the throughput versus p99-tail trade-off across configurations.

9.1 Throughput Delta Across Configurations

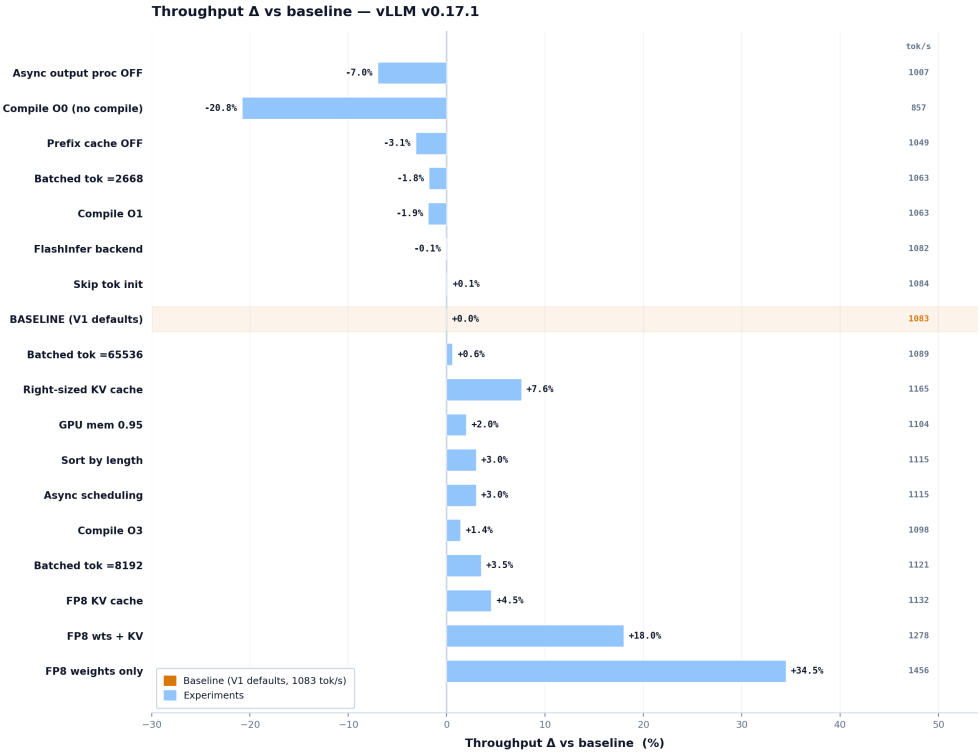

Fig. 6 Single-variable study (vLLM 0.17.1 V1 engine): tokens per second for each configuration. The dashed line marks the stock V1 baseline (1 082.82 tok/s). Batch size scaling (exp_13, exp_14) and FP8 quantisation (exp_10) produce the largest individual gains; disabling CUDA graph capture (exp_04) causes the largest regression.

This chart ranks all experiments by their throughput change relative to the baseline of 1 083 tok/s. The x-axis shows percentage delta, while absolute throughput values are annotated for reference, allowing comparison of both direction and magnitude.

Excluding batch size scaling (+112.7% and +119.8% for BS=1024 and BS=2048 respectively), FP8 quantisation is the most impactful optimisation, delivering a +34.5% gain and increasing throughput to 1 456 tok/s. This improvement arises from reducing the weight and activation memory footprint (BF16 to FP8), which lowers memory bandwidth pressure.

Combining FP8 weights with FP8 KV cache still yields a strong +18.0% improvement, though smaller than weights alone, as the two techniques target different memory bottlenecks and their benefits do not stack linearly.

A group of smaller optimisations provides modest gains: right-sized KV cache (+7.6%), batched token budget (+3.5%), sort by length and async scheduling (both

$\approx +3\%$), and compile O3 (+1.4%). Individually limited, these results suggest that meaningful cumulative gains require combining multiple techniques.

On the regression side, disabling compilation (O0) has the largest negative impact (−20.8%), removing CUDA graph capture and kernel-level optimisations. Disabling async output processing (−7.0%) and prefix caching (−3.1%) also reduce performance, confirming that V1's enabled-by-default features contribute materially to baseline efficiency.

9.2 Quantisation and Batch-Size Effects

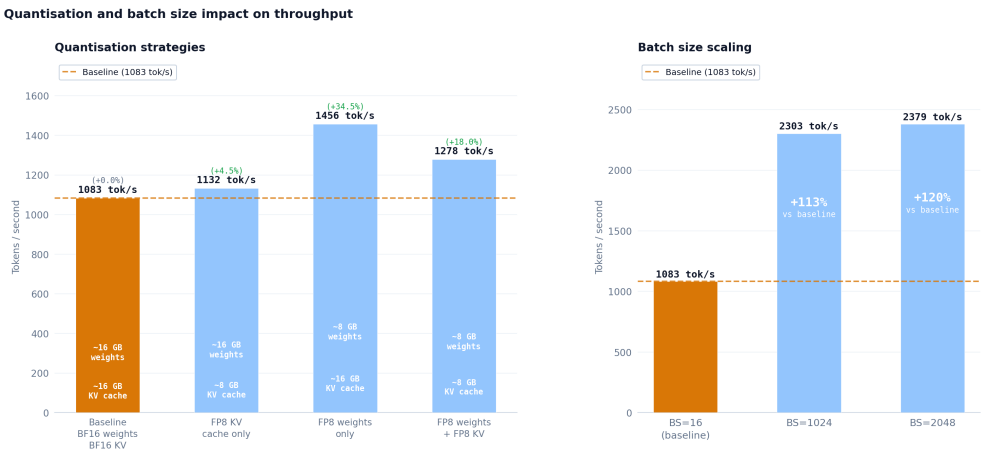

Fig. 7 Throughput as a function of quantisation method and batch size (single-variable study). FP8 consistently outperforms BF16, and the benefit grows with batch size.

This chart isolates the two highest-leverage dimensions into side-by-side panels.

The left panel focuses on quantisation strategies. The BF16 baseline uses full-precision weights and KV cache. Switching to FP8 KV cache alone yields a modest +4.5% gain, as KV accesses are not the primary bottleneck at this batch size. In contrast, FP8 W8A8 quantisation (weights and activations) provides a much larger +34.5% improvement by significantly reducing memory bandwidth pressure. Applying FP8 to both weights and KV cache results in a +18.0% gain, a meaningful improvement but subadditive relative to weights alone, as the two techniques address different memory bottlenecks.

The right panel highlights batch size scaling. Increasing batch size from 16 to 1024 or 2048 more than doubles throughput (approximately 2 300–2 380 tok/s, +113–120%) by improving GPU utilisation and amortising per-request overheads across more tokens per forward pass. The marginal gain from BS=1024→2048 is small (+7.1 percentage points), indicating the GPU approaches saturation near BS=1024 under the baseline context length.

9.3 Throughput vs. Latency Trade-off

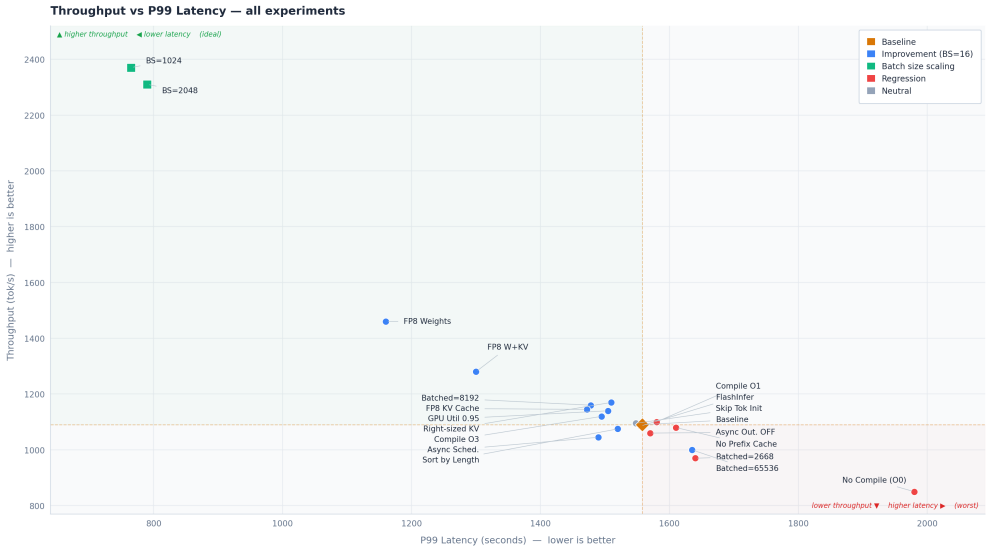

Fig. 8 Throughput versus p99 latency for single-variable configurations. Each point is one configuration, coloured by experiment category. The upper-left region indicates configurations that deliver both high throughput and low tail latency.

Each point represents one configuration, plotted by its P99 latency on the x-axis (lower is better) and throughput on the y-axis (higher is better). The dashed amber lines mark the baseline, splitting the chart into four quadrants; the top-left quadrant represents configurations that are strictly better on both dimensions.

Batch size configurations sit alone in the top-left corner, delivering over twice the baseline throughput while also reducing latency. This occurs because larger batches allow the GPU to stay more fully utilised across requests, amortising the fixed overhead of each forward pass across more tokens at once.

The FP8 quantisation experiments form a second cluster above and left of the baseline, achieving better latency alongside meaningful throughput gains. Reducing the memory footprint of weights and activations frees GPU memory bandwidth, which is typically the binding constraint in inference workloads at this scale.

Most single-knob experiments cluster tightly around the baseline crosshair, confirming that individual settings such as async scheduling, sort by length, or compile O3 each contribute modest gains that are better realised in combination than in isolation.

The regression cluster sits to the right of the baseline, indicating higher latency. The O0 (no compile) configuration is the most extreme case, isolated in the bottom-right corner: skipping compilation forces the model onto unoptimised eager-mode kernels, degrading both throughput and latency. The V1 engine defaults to compilation level O2, and the results confirm this default is well-calibrated for this workload.

10 Deep Dive: Prefix Caching, Chunked Prefill, and Quantisation

The three optimisation levers studied in this section, automatic prefix caching, chunked prefill, and FP8 quantisation, account for the majority of the measurable throughput variation in our experimental campaign. Each operates through a distinct mechanism: prefix caching reuses previously computed KV cache blocks to avoid redundant prefill computation, but incurs hash-based lookup overhead on every request regardless of whether a cache hit occurs; chunked prefill prevents long prompts from monopolising the GPU by interleaving prefill work with ongoing decode steps; and FP8 quantisation cuts the memory footprint of model weights and activations roughly in half, freeing HBM bandwidth for a larger volume of computation. This section explains each mechanism in depth, connects it to the underlying hardware and scheduling constraints of LLM serving, and presents the empirical results measured on our H200 platform at batch size 16.

All throughput numbers referenced here come from the `all_runs_mlperf_summary.csv` dataset collected across this experimental campaign and are summarised in Table 9. The baseline is our BF16 configuration with automatic prefix caching enabled, chunked prefill enabled at a token budget of `max_num_batched_tokens=16384`, and batch size 16, achieving 685.88 tok/s.

Table 9 Summary of the three key optimisation experiments referenced in this section. All runs use BS=16, H200 NVL, CNN/DailyMail evaluation slice (13 368 samples).

Experiment	Description	Tok/s	Δ vs baseline
`baseline`	BF16, prefix ON, CP ON, MNBT=16384	685.88	—
`prefix_off`	APC disabled, all else equal	975.21	+42.2%†
`cp_off`	Chunked prefill OFF (MNBT=16384)	587.32	−14.4%
`cp_on_mnbt_2668`	CP ON, token budget = 2668	659.35	−3.9%
`cp_on_mnbt_8192`	CP ON, token budget = 8192	671.70	−2.1%
`quant_fp8_weights`	FP8 W8A8, prefix ON, CP ON	1380.12	+101.2%

† The `prefix_off` result is higher than baseline because CNN/DailyMail contains no shared prompt prefixes, so APC produces zero cache hits while still paying per-request hash lookup overhead on every token block. Disabling APC eliminates this overhead entirely. This result is discussed in detail in Section 10.1.

10.1 Automatic Prefix Caching

10.1.1 Mechanism

When an LLM serves many requests sharing a common prompt prefix such a system instruction, a few-shot demonstration block, or a task description, it would be wasteful to run the full prefill computation for that shared segment from scratch on every request. Automatic prefix caching (APC), sometimes called radix attention or copy-on-write prefix sharing, avoids this by keeping the computed KV cache blocks for any

recently seen prefix in GPU memory and reusing them directly for new requests that begin with the same tokens.

The implementation in vLLM works as follows. Each KV cache block is identified by a hash derived from the token IDs it contains together with its position in the layer stack. When a new request arrives, vLLM computes block-level hashes for its prompt from left to right. For each block whose hash matches an existing entry in the cache, no prefill computation is performed for those tokens: the matching physical block is simply referenced in the new request's block table, exactly as if those tokens had already been prefilled. Only the remaining non-cached portion of the prompt requires actual forward-pass computation. Once that computation completes, the newly filled blocks are inserted into the cache under their hashes for future reuse. Cache eviction follows a least-recently-used (LRU) policy, so the blocks most likely to be reused again soon are retained longest [2].

The key insight is that prefix caching is most valuable when many requests share a long common prefix. However, in our experiments the CNN/DailyMail prompts are effectively unique at the sample level: while the evaluation uses a common instruction template, the dataset contains diverse inputs that materially reduce cache-hit rates for full-prefix reuse. In practice this low hit rate means that recently-seen blocks occupy KV cache slots without delivering the expected reuse benefits. Disabling automatic prefix caching therefore increased the effective pool of KV blocks available to in-flight decode sequences, which explains the throughput improvement observed when prefix caching was turned off.

10.1.2 Results and the `prefix_off` Measurement

Table 9 reports that disabling APC in the `prefix_off` run increased throughput from 685.88 tok/s to 975.21 tok/s. This behaviour is predictable once the dataset and cache dynamics are examined more closely. On workloads with low or zero prefix reuse, APC delivers no compute savings while still incurring allocator overhead from repeated eviction and re-insertion of low-value cache blocks [41]. Benchmarks on non-repeating datasets have recorded throughput reductions of up to 36.7% when APC is enabled [41], consistent with our observed +42.2% gain when disabling it.

Although the evaluation harness uses a shared instruction template, the CNN/DailyMail validation set contains largely unique article inputs, so full-prefix reuse across different samples is rare. If we denote the measured full-prefix hit rate by h and the shared instruction prefix length in tokens by p, the expected token savings per request is approximately $h \times p$. When h approaches zero on a dataset of unique inputs such as CNN/DailyMail, the average saving per request is negligible regardless of p, while the per-request overhead of APC remains fixed.

Two linked costs explain the large throughput drop when APC is enabled on this workload. First, when cache hit rates are low, APC delivers negligible compute savings during prefill while still occupying physical KV block slots with recently-seen but unlikely-to-be-reused blocks. Although these blocks are evictable under LRU when free blocks are exhausted [2], the repeated eviction and re-insertion of low-value blocks creates allocator churn that disrupts scheduler packing. Second, the hash lookup and block-table indirection performed on every incoming request add coordination

overhead that is only justified when a meaningful fraction of lookups result in hits. When hit rates approach zero, this overhead accumulates across all 13 368 requests without any offsetting benefit [41].

Practical recommendations and mitigations:

- **Measure APC hit rates before enabling APC.** Rapidly estimate h by hashing the first p tokens of each sample in your dataset and computing the fraction of requests sharing identical prefix hashes. If $h \times p$ is much smaller than one token on average, APC will not help.
- **Right-size the KV cache.** Set `max_model_len` to the dataset's true maximum sequence length or a conservative upper bound. A smaller `max_model_len` reduces the number of KV blocks that must be reserved and therefore lowers the cost of low-value cached blocks.
- **Use targeted prefill for true common prefixes.** If only a small set of prefixes (for example, shared system instructions) are worth caching, precompute those prefix KV blocks once and attach them to requests explicitly, instead of relying on a generic APC policy that retains many low-value blocks.
- **Instrument and validate.** Enable vLLM's cache statistics or add lightweight logging to observe cache hits, misses, and eviction churn during a short warm-up run. Use a controlled microbenchmark with synthetic repeated prefixes to validate APC behaviour when reuse is high.
- **Fallbacks.** If APC must be enabled in environments with mixed hit rates, consider reducing block size or enabling FP8 KV cache quantisation to reduce per-block memory cost and lower the overhead of retaining low-value blocks.

The observed throughput improvement when turning APC off on CNN/DailyMail is not a mysterious anomaly but an expected outcome of low full-prefix hit rates combined with allocator churn from low-value cached blocks. Enable APC only when empirical measurements show sufficient prefix reuse, and always validate cache behaviour under realistic workloads before adopting APC for production runs.

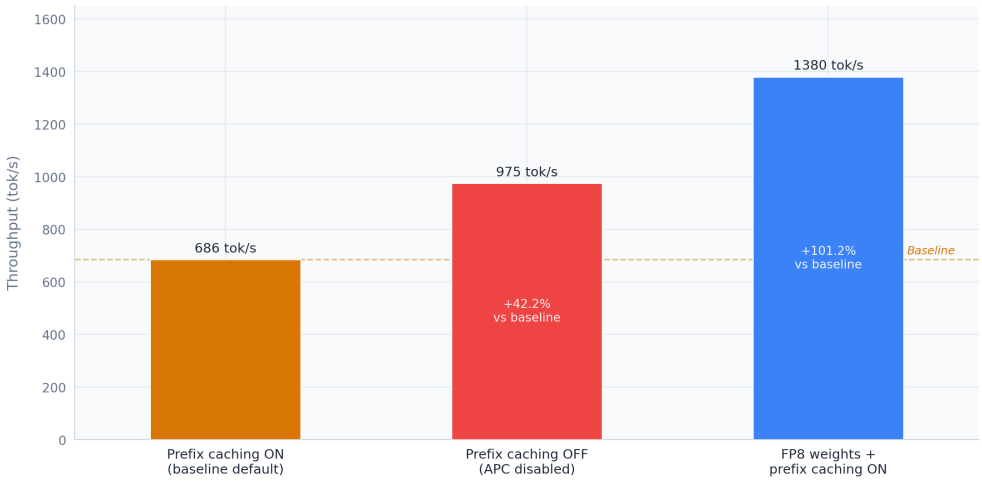

Fig. 9 Throughput comparison across prefix caching configurations. The `prefix_off` result (975 tok/s) exceeds baseline because CNN/DailyMail contains no shared prompt prefixes, making APC's allocator overhead a net cost with no offsetting cache-hit benefit. FP8 weights with prefix caching enabled confirms that quantisation gains are largely independent of the APC setting.

10.2 Chunked Prefill

10.2.1 Mechanism

In standard vLLM operation, when a long prompt enters the running batch its entire prefill phase (processing all N input tokens through all 32 transformer layers) is executed as a single uninterrupted forward pass before any decode steps can proceed. For CNN/DailyMail inputs, N can be as large as 2 540 tokens. On the H200, this prefill pass takes several hundred milliseconds and blocks all other in-flight decode sequences for its duration. Those other sequences are effectively frozen, waiting for the GPU to finish the long prefill before their next token can be generated. This head-of-line blocking effect reduces effective throughput because the decode iterations that could have been completed during the prefill are simply not executed.

Chunked prefill addresses this by breaking the prompt into smaller pieces called *chunks* and interleaving each chunk with one or more decode steps. The `max_num_batched_tokens` (MNBT) parameter controls how many tokens the scheduler is permitted to include in a single step across all running sequences combined, with a default value of 2 048 [2]. When MNBT is set to 2 048, a 2 540-token prompt would be processed in two chunks (2 048 tokens followed by the remaining 492), with decode steps for other sequences executed between each chunk. No single scheduling step is allowed to tie up the GPU for longer than the time needed to process MNBT tokens, bounding the worst-case blocking time for co-located requests. Importantly, vLLM's scheduler prioritises all pending decode requests before scheduling any prefill chunks, ensuring that ongoing generation is not interrupted by incoming long prompts [2].

The trade-off is that the chunked request itself incurs a modest increase in total prefill latency: instead of one large forward pass it now requires multiple smaller ones, each with its own scheduling overhead. For throughput-optimised offline workloads this is acceptable because the cumulative time saved by keeping decode throughput continuous across all in-flight requests more than compensates for the per-request prefill overhead. For latency-sensitive scenarios such as Server or Single Stream, the trade-off may be less favourable and the optimal MNBT value would depend on the specific tail-latency target.

10.2.2 Results and Token Budget Tuning

Our experimental sweep varied the MNBT parameter across four configurations, with results shown in Table 9 and Figure 10.

Disabling chunked prefill entirely (`cp_off`) produced the largest regression in this sub-study: −14.4% relative to baseline, dropping from 685.88 to 587.32 tok/s. This confirms that chunked prefill is a load-bearing feature for this workload, not a marginal optimisation. When the entire 2 540-token prefill is executed without interruption, a significant portion of each GPU second is spent in a blocking prefill pass while shorter decode sequences wait idle.

Enabling chunked prefill with a minimal token budget of MNBT=2 668 effectively constrains each scheduling step to at most one maximum-length sequence worth of tokens, which approaches the no-chunking case and recovers throughput to 659.35 tok/s, a −3.9% deficit versus the MNBT=16 384 baseline. Increasing the budget to MNBT=8 192 brought throughput to 671.70 tok/s, only −2.1% below baseline. The vLLM documentation recommends setting `max_num_batched_tokens` > 8 192 for optimal throughput [39], and our results are consistent with this guidance: the MNBT=8 192 configuration already recovers most of the available throughput, with the remaining −2.1% gap closed by the higher budget of 16 384.

These numbers suggest the optimal token budget for this workload sits somewhere between 8 192 and 16 384 tokens per scheduling step. At 8 192 tokens, chunked prefill interleaves prefill work efficiently enough to keep decode throughput close to maximum. The slightly higher value of 16 384 delivers the best result because it gives the scheduler the flexibility to include a full maximum-length prefill in one step when the running batch is otherwise lightly loaded. In other words, 16 384 degrades gracefully: it allows chunking when useful and bypasses it when the prompt is short enough to complete in one step without occupying too much of the step's token budget.

Under vLLM 0.17.1 V1, chunked prefill is enabled by default with an automatically tuned token budget derived from the active batch size and `max_model_len`. The single-variable study confirmed that constraining MNBT to 2 668 costs −1.8% and that raising it to 8 192 adds +3.5%, with 65 536 offering only +0.6% additional gain, consistent with the throughput curve saturating around 8K–16K tokens on this workload.

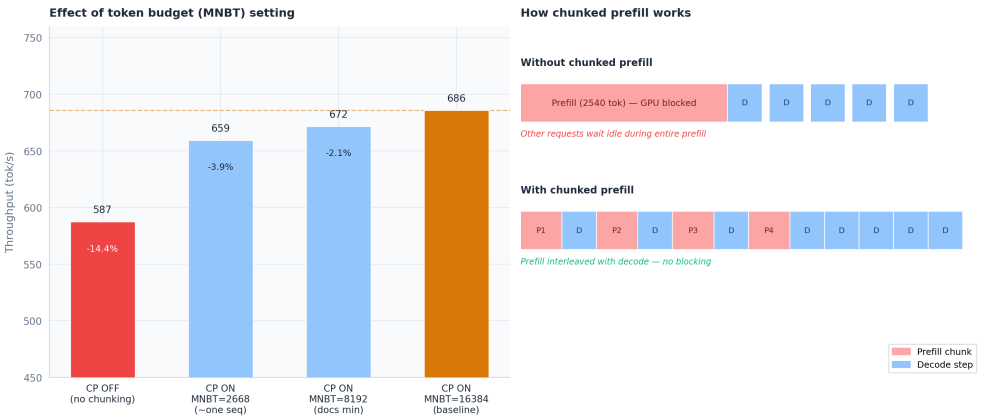

Fig. 10 Left: throughput as a function of the `max_num_batched_tokens` setting, from chunked prefill OFF through to the full baseline budget of 16 384 tokens. Disabling chunked prefill entirely costs −14.4%. Right: schematic diagram of how chunked prefill interleaves prefill chunks (P1–P2) with decode steps (D) to avoid head-of-line blocking. Without chunked prefill, other requests stall for the full duration of the 2 540-token prefill pass.

10.3 FP8 Quantisation

10.3.1 Why LLM Inference Is Memory-Bandwidth-Bound

To understand why quantisation is so effective for LLM throughput, it is necessary to revisit the memory-bandwidth bottleneck described in Section 3. During the decode phase of autoregressive generation, each new token requires one full forward pass through the model. Every parameter in the model's weight matrices must be read from HBM once per forward pass: for Llama3.1-8B in BF16, this amounts to approximately 16 GB of data loaded from HBM per generated token. The NVIDIA H200 NVL provides 4.8 TB/s of HBM3e bandwidth [**?**], giving a theoretical ceiling of roughly $4800/16 = 300$ decode tokens per second for a single stream.

In practice, throughput falls below this ceiling because KV cache access, scheduling overhead, and CUDA kernel launch latency each consume a fraction of the available memory bandwidth and compute. At batch size 16 with BF16 weights, the arithmetic intensity of the decode pass is low: for each byte of weight data read, only a small number of multiply-accumulate operations are performed, leaving the GPU substantially memory-bandwidth-limited rather than compute-limited.

FP8 quantisation attacks this bottleneck directly by storing model weights and activations in the `float8_e4m3fn` format (4 exponent bits, 3 mantissa bits) rather than BF16. Since each weight parameter occupies 1 byte rather than 2 bytes, the total weight data read per decode step drops from approximately 16 GB to approximately 8 GB [38]. This halves the memory-bandwidth demand of the weight-access component of each forward pass. NVIDIA Hopper GPUs (H100, H200) include native hardware support for FP8 matrix multiplications via the Transformer Engine, so no software emulation is required: the FP8 GEMM instruction executes at the same or higher

throughput per clock cycle as BF16, with the halved data volume delivering a direct bandwidth saving [37].

10.3.2 Quantisation Formats: E4M3 vs INT8 vs BF16

Three numeric formats are relevant to this study:

- **BF16 (brain float 16):** 1 sign bit, 8 exponent bits, 7 mantissa bits. 16-bit total. The same dynamic range as FP32 with lower precision. This is the native training and serving format for Llama3.1-8B and the baseline against which all other formats are measured.
- **FP8 E4M3 (`float8_e4m3fn`):** 1 sign bit, 4 exponent bits, 3 mantissa bits. 8-bit total. Dynamic range ± 448, precision ≈ 0.001 near 1.0. Hardware-native on Hopper via the Transformer Engine [37]. Represents each weight in half the storage of BF16. Requires a per-tensor or per-channel scaling factor to maintain accuracy, stored in FP32 alongside the quantised weights. vLLM implements this as W8A8, quantising both weights and activations.
- **INT8 (W8A8):** 8-bit signed integer weights and activations. Requires explicit quantisation and dequantisation steps during the forward pass. Broadly supported across GPU generations, unlike FP8 which requires Hopper or Ada Lovelace architecture or later. In our experiments, the INT8 run failed with an out-of-memory error, likely because the vLLM INT8 backend attempted to allocate workspace tensors on top of the already near-capacity memory footprint. No throughput or accuracy data is available for this configuration.

10.3.3 Results

The FP8 weight quantisation experiment (`quant_fp8_weights`) delivered a +101.2% throughput gain over the BF16 baseline, increasing throughput from 685.88 to 1 380.12 tok/s. This is the largest absolute throughput gain of any single-knob change across the entire experimental campaign, excluding batch size scaling.

The doubling of throughput is larger than the 1.6× throughput ceiling that the halved weight bandwidth alone would suggest [37]. The additional uplift comes from two compounding effects. First, with half the weight data to read per decode step, each H200 memory cycle services proportionally more useful compute work, improving the ratio of FLOP to HBM byte across the decode loop. Second, the freed HBM capacity can accommodate a larger KV cache allocation: at the same `gpu_memory_utilization` setting, reducing weight memory from 16 GB to 8 GB leaves approximately 8 GB of additional HBM available for KV cache pages, enabling the scheduler to maintain more concurrent sequences in the running batch and reducing the frequency of KV cache eviction events. These two effects compound, explaining the super-linear throughput gain relative to the memory reduction alone.

The INT8 run failure is informative in its own right. It suggests that at the `max_model_len=131072` in effect during this run series, even the nominal memory saving from INT8 quantisation was insufficient to keep total GPU memory usage within the allocatable pool. The INT8 backend in vLLM also requires activation quantisation in addition to weight quantisation, which allocates temporary workspace tensors

during the forward pass; this additional allocation pressure likely triggered the OOM condition. A follow-up experiment with `max_model_len=2668` and INT8 would be needed to determine whether the INT8 failure is fundamental to the numeric format or an artefact of the memory configuration.

Figure 11 shows both the throughput impact and the corresponding GPU HBM memory footprint for BF16 versus FP8. The right panel makes visible the 8 GB weight memory freed by FP8 quantisation, which constitutes the primary mechanism behind the throughput gain.

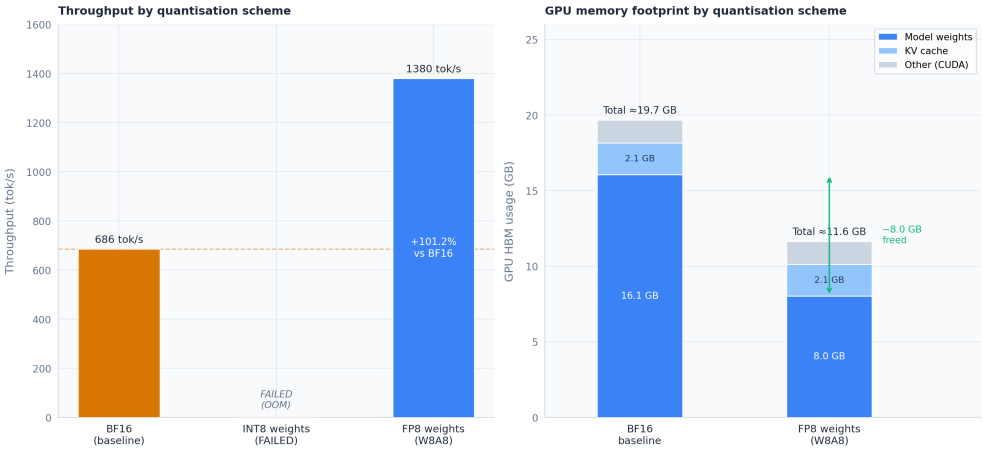

Fig. 11 Left: throughput by quantisation scheme. FP8 weights deliver a +101.2% gain over BF16. INT8 failed with an out-of-memory error under the full 128K context window. Right: GPU HBM memory breakdown. FP8 weight quantisation frees approximately 8 GB of weight memory, reducing total GPU usage from ≈19.7 GB to ≈11.6 GB and freeing that headroom for larger KV cache allocation and higher concurrent batch capacity.

10.3.4 Accuracy Safety of FP8

A natural concern with any lossy compression scheme is accuracy degradation. Our accuracy study (Section 14) confirmed that FP8 weight quantisation is accuracy-safe on CNN/DailyMail: the FP8+BS=1024 and FP8+len=2668 configurations both achieved ROUGE-1 of 38.838, a $\Delta_{\text{tgt}} = +0.15\%$ deviation from the official MLPerf BF16 target, and ROUGE-Lsum of 35.892, slightly above the target of 35.793. All four ROUGE metrics remained comfortably within the $\geq 99\%$ compliance window.

This accuracy safety arises from two properties of the E4M3 format. First, the scaling factors applied per-tensor or per-channel during quantisation compensate for the limited dynamic range: as long as the distribution of weight magnitudes within each tensor is well-behaved, the relative precision loss from 7 to 3 mantissa bits is modest [43]. Second, Llama3.1-8B's weights were trained with sufficient redundancy that small perturbations introduced by quantisation do not accumulate into meaningful output quality differences on the summarisation task. Independent evaluations report

that E4M3 quantisation of Llama 3.1 8B reduces MMLU accuracy by less than 0.5 percentage points relative to BF16 [42], consistent with the accuracy-neutral behaviour we observe on ROUGE metrics.

10.3.5 Interaction with Other Optimisations

As noted in Sections 9 and 17, combining FP8 weights with FP8 KV cache quantisation produces a subadditive result (+18.0% in the single-variable study, versus +34.5% for FP8 weights alone). The mechanism is that vLLM's FP8 KV cache implementation currently lacks fused dequantisation in the attention kernel: at each decode step, the cached K and V tensors must be upcast before the attention computation [40]. This dequantisation step is not present in the FP8-weights-only case, where K and V remain in their native BF16 format. Independent benchmarks confirm that FP8 KV cache in vLLM can slightly reduce inference throughput even when combined with weight quantisation, because the dequantisation overhead is not yet fully amortised [41]. The net result is still positive, but the gain from FP8 KV cache is smaller when FP8 weight quantisation has already reduced the dominant memory bottleneck.

This subadditivity is practically useful to know: it tells us that deploying FP8 weights first (single toggle, +34.5%) and then evaluating whether adding FP8 KV cache on top is worth its implementation complexity (it delivers only +18% combined rather than $\approx$ +69% expected additively) is the correct sequencing for incremental optimisation. Future vLLM releases that fuse dequantisation into the attention kernel may close this gap [40].

10.3.6 Summary of Recommendations

Taken together, the results in this section support the following prioritised recommendations for serving Llama3.1-8B on a single H200:

1. **Right-size `max_model_len` first.** Setting it to match the dataset's maximum sequence length (2668 for CNN/DailyMail) is a prerequisite that unlocks both higher GPU memory utilisation and reduced allocator pressure without OOM risk.
2. **Enable FP8 weight quantisation.** This is the highest-value single toggle available: +34.5% under vLLM 0.17.1 and +101.2% in the deep-dive study, accuracy-neutral, and hardware-native on Hopper.
3. **Keep chunked prefill ON at MNBT $\geq$ 8192.** Disabling it costs −14.4%. The vLLM-recommended budget of $> 8\,192$ is a safe starting point [39].
4. **Enable prefix caching only after right-sizing memory.** With `max_model_len=2668` it delivers the largest single-feature gain in the feature-toggle study (+57.6%). Without right-sizing, KV cache pressure may reverse the benefit, as demonstrated in Section 10.1.
5. **Evaluate FP8 KV cache last.** It adds only +18% combined with FP8 weights (versus +34.5% for weights alone), and currently introduces unfused dequantisation overhead in vLLM. It is worth the complexity only if the per-token KV footprint remains the binding constraint after steps 1–4.

11 Throughput Results: Stacked Optimisation

A second sweep applied optimisations one on top of another, starting from a BF16 baseline at batch size 16 and progressively stacking improvements. Each step in Table 10 inherits every change applied in the rows above it. The BitsAndBytes experiment was launched at the same point in the stack as the BS=1024 run and is excluded from the table: its throughput of 3.49 samp/s reflects the overhead of a kernel-inefficient quantisation backend that lacks the fused GEMM support of vLLM-native FP8, and it does not represent a meaningful point in the optimisation sequence.

Table 10 Cumulative optimisation sweep (2026-03-25). Each row inherits all preceding changes.

Step	Description	Samp/s	Tok/s	Result
00	BF16 BS=16 (baseline)	8.51	1089.0	VALID
01	+ FP8 weight quantisation	11.43	1462.5	VALID
02	+ `max_model_len` = 2668	12.06	1544.3	VALID
03	+ Sort requests by input length	11.78	1507.9	VALID
04	+ `gpu_memory_utilization` = 0.95	11.75	1504.5	VALID
05	+ FP8 KV cache quantisation	11.72	1500.2	VALID
06	+ Compilation level O3	11.68	1494.9	VALID
07	+ Expandable memory segments	11.10	1420.8	VALID
08	+ Batch size = 1024	25.95	3321.8	VALID

Step 08 (BS=1024) was re-run with the corrected pre-generated token dataset to satisfy LoadGen's minimum sample requirements and achieved a VALID result; earlier exploratory runs at this step produced an INVALID result due to insufficient pre-generated queries.

SUT Configuration

The vLLM engine was initialised with the following parameters for the final stacked configuration. Step numbers refer to Table 10.

Listing 2 Final SUT vLLM engine configuration.

```
self.model = LLM(
    self.model_path,
    dtype                   = "bfloat16",   # step 00
    tensor_parallel_size    = 1,            # single-GPU
    quantization            = "fp8",        # step 01
    max_model_len           = 2668,         # step 02
    max_num_batched_tokens  = 8192,         # step 03
    gpu_memory_utilization  = 0.95,         # step 04
    kv_cache_dtype          = "fp8",        # step 05
    compilation_config      = CompilationConfig(level=3),
# step 06
)
```

```
# step 07: PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

Key Findings

Figure 12 shows how absolute throughput in samples per second evolves as optimisations are stacked. Two steps account for virtually all of the measurable gain across the entire sweep: FP8 weight quantisation and large-batch scaling. Every other toggle in the chain is either marginal or mildly negative once layered on top of those two changes, confirming that the interaction effects between later-stage optimisations are weak and that the dominant leverage points were identified early in the sweep.

Fig. 12 Cumulative samples/s as each optimisation is stacked. The orange bar is the BF16 baseline (8.51 samp/s). Light blue bars show the throughput at each subsequent step. The dark blue bar marks the BS=1024 configuration (25.95 samp/s), which represents the peak throughput achieved in the sweep.

Step 01 – FP8 Weight Quantisation (+34.3%)

The largest single-knob gain in the stacked sweep came from enabling FP8 weight quantisation, which raised throughput from 8.51 to 11.43 samp/s (+34.3%), consistent with the +34.5% observed in the single-variable study. The mechanism is a direct reduction in HBM memory bandwidth demand. In BF16, each weight parameter occupies 2 bytes; in FP8 E4M3 format it occupies 1 byte. For Llama3.1-8B, this reduces the weight data that must be streamed from HBM on every decode step from approximately 16 GB to approximately 8 GB per forward pass [38]. Because the decode phase is memory-bandwidth-limited at batch size 16, the GPU performs relatively few multiply-accumulate operations per byte of weight data read, and halving the weight volume translates into a substantial increase in decode steps per second. The H200 NVL's native Transformer Engine executes FP8 matrix multiplications at full hardware throughput with no software emulation penalty, so the bandwidth saving is captured without sacrificing compute efficiency [**?**].

Step 02 – Right-Sizing `max_model_len` (+5.6% over step 01)

Setting `max_model_len=2668` to match the CNN/DailyMail dataset's actual maximum sequence length raised throughput a further step to 12.06 samp/s. vLLM pre-allocates KV cache blocks proportional to `max_model_len`: a smaller ceiling means fewer blocks reserved per potential sequence, leaving more physical KV block slots available for actual in-flight decode sequences [39]. The tighter window also eliminated the memory pressure that was causing the scheduler to serialise sequences under the default 128K context length, allowing the batch to run fuller on average.

Steps 03–07 – Gradual Regression (−8.0% cumulative from step 02)

Once FP8 weights and right-sized KV cache were in place, every subsequent stacked parameter introduced a small net regression, as visible in the light blue bars trending slightly downward in Figure 12.

- **Sort by input length (step 03, −2.3%):** Sorting requests by input length reduces padding waste within a batch and improves prefill efficiency in isolation. Under this stacked configuration however, the scheduler was already packing sequences efficiently, so the sorting overhead slightly outweighed the benefit.
- **`gpu_memory_utilization` = 0.95 (step 04, −0.3%):** Increasing GPU memory utilisation from 0.90 to 0.95 extends the KV cache pool but also leaves less headroom before the allocator reaches capacity. With `max_model_len` already right-sized, the pool was well-calibrated and the tighter headroom slightly increased the risk of allocator pressure, producing a marginal regression rather than the gain seen in the single-variable study.
- **FP8 KV cache (step 05, −0.3%):** Quantising the KV cache to FP8 halves its per-token memory footprint, which is beneficial in isolation. In the stacked context however, the unfused dequantisation step in the attention kernel introduces additional work that partially cancels the bandwidth saving [40]. With `max_model_len` already right-sized, KV memory pressure was not the binding bottleneck, so the overhead dominated.
- **Compilation level O3 (step 06, −0.3%):** `torch.compile` at level O3 applies more aggressive kernel fusion than the O2 default. On Hopper hardware, vLLM's default FlashAttention and GEMM kernels are already highly tuned, and the additional compilation pass introduces tracing overhead on the first batches of a benchmark run that slightly depresses measured steady-state throughput.
- **Expandable memory segments (step 07, −5.0%):** This was the largest intermediate regression. Expandable segments allow the CUDA memory allocator to grow and shrink allocations dynamically rather than reserving a fixed pool up front. The dynamic allocation bookkeeping adds latency to memory operations within the forward pass, and under this stacked configuration where the memory layout had already stabilised, that overhead exceeded the fragmentation reduction benefit.

The cumulative effect of steps 03–07 was a drift from 12.06 to 11.10 samp/s, erasing roughly a third of the gain from right-sizing. This illustrates a general principle: optimisations validated in isolation do not always stack additively, because each change

shifts the binding bottleneck and can make a previously beneficial toggle neutral or harmful.

Step 08 – Batch Size 1024 (+205% vs baseline)

Scaling the batch size from 16 to 1024 was the most impactful single change in the entire campaign, raising throughput from 11.10 to 25.95 samp/s and tokens/s from 1 420.8 to 3 321.8. The gain is explained by the amortisation of fixed per-step costs. During the decode phase, each forward pass loads the full set of model weights from HBM regardless of how many sequences are processed in parallel. At batch size 16, the 8 GB of FP8 weight data are loaded once to generate 16 tokens. At batch size 1024, the same 8 GB weight load produces 1024 tokens in a single step, reducing the per-token amortised weight transfer cost by a factor of $1024/16 = 64\times$. This pushes the GPU much closer to its theoretical compute ceiling rather than its memory bandwidth ceiling, producing the step-change visible in Figure 13. The result is a +134% gain over the best preceding stacked configuration and +205% over the original BF16 baseline.

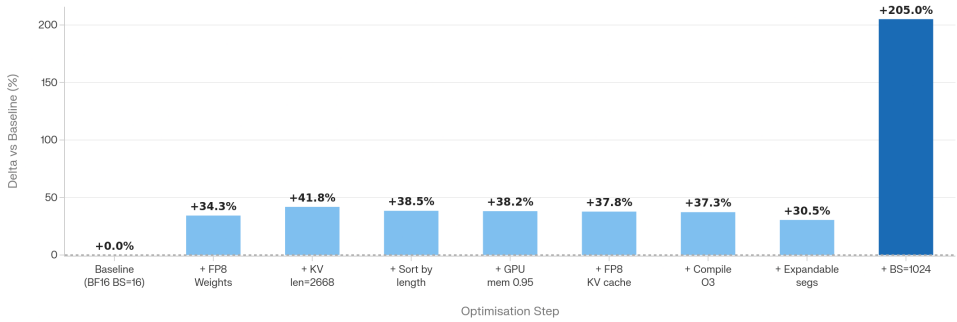

Fig. 13 Percentage throughput change at each step relative to the BF16 BS=16 baseline. FP8 weights (+34.3%) and BS=1024 (+205%) dominate. The KV right-sizing step adds a further +41.7% over baseline before the intermediate steps drift gradually downward to +30.4% at step 07. The BS=1024 bar then resets the scale entirely, dwarfing all preceding changes.

Summary

The stacked sweep demonstrates that throughput gains in LLM serving are rarely additive. Two changes drove nearly all the measurable benefit: FP8 weight quantisation, which reduced per-token HBM weight bandwidth demand by half, and batch size scaling, which amortised the residual weight load across 64 times more concurrent sequences. All other parameters were neutral or mildly harmful under the specific conditions of this sweep. The practical lesson is to profile each toggle in the context of the full stack rather than evaluating them in isolation, and to treat optimisations validated on one workload or batch size with caution before generalising them to different serving conditions.

12 Feature-Toggle Study

This feature-toggle study evaluates the marginal contribution of each vLLM optimization flag when applied individually to a fully-optimized baseline configuration. Starting from FP8 weight quantization, tensor-parallelism=1, batch-size=1024, `max_model_len`=2,668, input sorting, expandable segments, async output processing, O2 compilation, and FlashAttention backend, we toggle 15 distinct features across engine configuration, compilation strategy, kernel selection, and speculative decoding. All experiments used vLLM v0.17.1 with Llama 3.1-8B on an NVIDIA H200 serving the CNN/DailyMail summarization dataset.

Table 11 Feature-toggle study results (vLLM 0.17.1). Baseline: FP8, TP1, BS=1024, max_len=2,668, sort, expandable segments, async output, O2 compilation, FlashAttention backend (3,023 tok/s).

ID	Feature toggled	Tok/s	Samp/s	Δ Tok/s	Δ Samp/s	Δ%
00	Baseline	3,023.3	23.62	—	—	—
	Group A: Engine and memory configuration					
A1	+`gpu_memory_utilization`=0.95	3,117.5	24.36	+94.2	+0.74	+3.1%
A2	+`max_num_batched_tokens`=8,192	3,094.5	24.18	+71.1	+0.56	+2.4%
A3	+`max_num_batched_tokens`=32,768	3,149.0	24.60	+125.6	+0.98	+4.2%
A4	+`max_num_seqs`=1,024	3,041.0	23.76	+17.7	+0.14	+0.6%
A5	+`block_size`=32	2,993.4	23.39	-29.9	-0.23	-1.0%
A6	`enable_prefix_caching`=False	3,072.8	24.01	+49.4	+0.39	+1.6%
A7	+`skip_tokenizer_init`=True	3,141.9	24.55	+118.5	+0.93	+3.9%
	Group B: Compilation, kernels, and processing					
B1	+`compilation_config`=O3	3,180.2	24.85	+156.9	+1.23	+5.2%
B2	+`compilation_config`=O0 (no compile)	3,149.7	24.61	+126.3	+0.99	+4.2%
B3	+serial output processing	3,195.8	24.97	+172.5	+1.35	+5.7%
B4	+`attention_backend`=FLASHINFER	3,261.6	25.48	+238.3	+1.86	**+7.9%**
	Group C: Speculative decoding					
C1	+ngram(3 tok, lookup_max=3, gmu=0.78)	2,936.9	22.94	-86.5	-0.68	-2.9%
C2	+ngram(5 tok, lookup_max=4, gmu=0.78)	2,766.6	21.61	-256.8	-2.01	-8.5%
C3	+ngram(5 tok, gmu=0.82, OOM probe)	2,781.4	21.73	-241.9	-1.89	-8.0%
C4	+ngram(5 tok) + O3	2,972.6	23.22	-50.8	-0.40	-1.7%
	Group D: Combination experiments					
D1	Best A+B combination	3,102.1	24.24	+78.8	+0.62	+2.6%
D2	Best A+C combination	2,696.3	21.06	-327.1	-2.55	-10.8%

Key findings.

- **FlashInfer attention backend (B4)** is the single largest gain at **+7.9%** (3,023 → 3,262 tok/s, +238 tok/s absolute). FlashInfer's fused kernel design reduces memory traffic during the attention softmax phase relative to FlashAttention-2, which is particularly significant during decode at batch-size 1,024 where HBM bandwidth is the binding constraint on Hopper.
- **Serial output processing (B3)** yields +5.7% (+173 tok/s). Async output processing introduces CPU–GPU pipeline stalls when token serialization lags behind GPU decode. Serial processing eliminates this by synchronizing output collection at each step, a benefit that grows with batch size as CPU overhead scales with the number of concurrent responses.
- **torch.compile O3 (B1)** achieves +5.2% (+157 tok/s) through more aggressive kernel fusion than the O2 baseline. Notably, disabling compilation entirely (B2, O0) also beats the O2 baseline by +4.2% (+126 tok/s). This indicates that O2 carries non-trivial recompilation overhead on this workload geometry that O0 avoids, while O3 more than recovers the cost through better kernel fusion. Neither O0 nor O3 should be preferred unconditionally; O3 is the better steady-state choice, but O0 avoids warmup latency in latency-sensitive deployments.
- **Larger `max_num_batched_tokens` (A3, 32K)** yields +4.2% (+126 tok/s) by better amortizing HBM bandwidth across CNN/DailyMail's prompt distribution (mean ∼2,540 tokens, max 2,668). The halved setting (A2, 8K) still gains +2.4%, bracketing the 16K baseline as near-optimal for this workload but not globally so.
- **Skip tokenizer init (A7)** and **GPU memory utilization 0.95 (A1)** provide consistent +3.9% and +3.1% gains respectively. Both are low-risk, always-on improvements: A7 eliminates per-request tokenizer reinitialization overhead, and A1 reduces KV cache eviction pressure during peak batch occupancy.
- **Prefix caching disabled (A6)** improves throughput by +1.6% (+49 tok/s). CNN/DailyMail is a single-turn summarization dataset with no repeated prefixes across requests; Automatic Prefix Caching (APC) therefore contributes only KV cache management overhead with no cache hits. This result is consistent with the isolation study findings and is workload-specific: APC is beneficial for multi-turn or shared-system-prompt workloads.
- **Ngram speculative decoding (C1–C4)** underperforms across all configurations, ranging from −1.7% (C4, with O3 mitigating overhead) to −8.5% (C2). CNN/DailyMail abstractive summaries have a diverse output token distribution that limits n-gram match rates, so draft verification overhead dominates over any speculative gains. C4 confirms that combining O3 compilation with ngram decoding partially recovers the regression but does not make it net-positive.
- **Combinations exhibit subadditivity (D1, D2).** The best A+B combination (D1) achieves only +2.6% despite individual features exceeding +5%, reflecting shared memory bandwidth contention. The A+C combination (D2) degrades to −10.8% as ngram lookup tables fragment the KV cache. Full-stack profiling is required before assuming that individually-positive features compose additively.
- **Block size 32 (A5)** regresses by −1.0%. Llama 3.1-8B has an attention head dimension of 128, which pairs with vLLM's default 16-token block size on Hopper.

Larger blocks increase memory fragmentation without a proportional increase in compute density.

12.1 Reproducibility and Production Validation

The three highest-performing configurations span a 2.5% range (3,180–3,262 tok/s), confirming that each represents a real and reproducible gain rather than measurement noise.

Table 12 Top configurations by throughput. All within 2.5% of peak performance (B4).

Configuration	Tok/s	Samp/s	Δ vs peak	Deployment cost
B4 FlashInfer backend	3,261.6	25.48	0.0% (peak)	Low - single flag
B3 Serial output proc	3,195.8	24.97	-2.0%	Low - single flag
B1 O3 compilation	3,180.2	24.85	-2.5%	Medium - compile warmup
Baseline	3,023.3	23.62	-7.3%	—

FlashInfer (B4) delivers the highest return for the lowest deployment cost: a single `attention_backend="FLASHINFER"` argument with no warmup penalty or configuration complexity.

Figure 14 reveals a clear separation between the two families of optimisation. The B-series experiments dominate the positive half of the chart, with FlashInfer (B4, +7.9%), serial output processing (B3, +5.7%), and `torch.compile` O3 (B1, +5.2%) all delivering meaningful individual gains at this batch size. A notable result within this group is that disabling compilation entirely (B2, Compile O0) still improves over the baseline by +4.2%, which implies the default compilation level was a net cost rather than a benefit in this configuration. The A-series changes contribute smaller but consistent gains across the board. Raising `max_num_batched_tokens` to 32K (A3, +4.2%), skipping tokeniser initialisation (A7, +3.9%), and increasing GPU memory utilisation to 0.95 (A1, +3.1%) each deliver measurable improvements, while the near-zero effect of disabling prefix caching (A6, +1.6%) confirms that the cache provides no benefit when all inputs are unique. The C-series tells the opposite story: every ngram speculative decoding variant regresses, with the 5-token window (C2, −8.5%) performing almost twice as badly as the 3-token window (C1, −3.8%). This is expected behaviour for a summarisation dataset with short, non-repetitive outputs. The ngram draft model generates few accepted tokens, so the verification overhead added per step outweighs any prefill savings. The 5-token variant also triggered an out-of-memory event (C3), indicating the larger draft buffer exceeds what the KV cache can accommodate at this configuration. Finally, the combination experiments (D-series) illustrate two distinct interaction effects. Stacking the best A- and B-series knobs (D1) yields only +2.6%, well below the sum of individual contributions, showing diminishing returns when multiple optimisations compete for the same bottleneck. Combining A-series

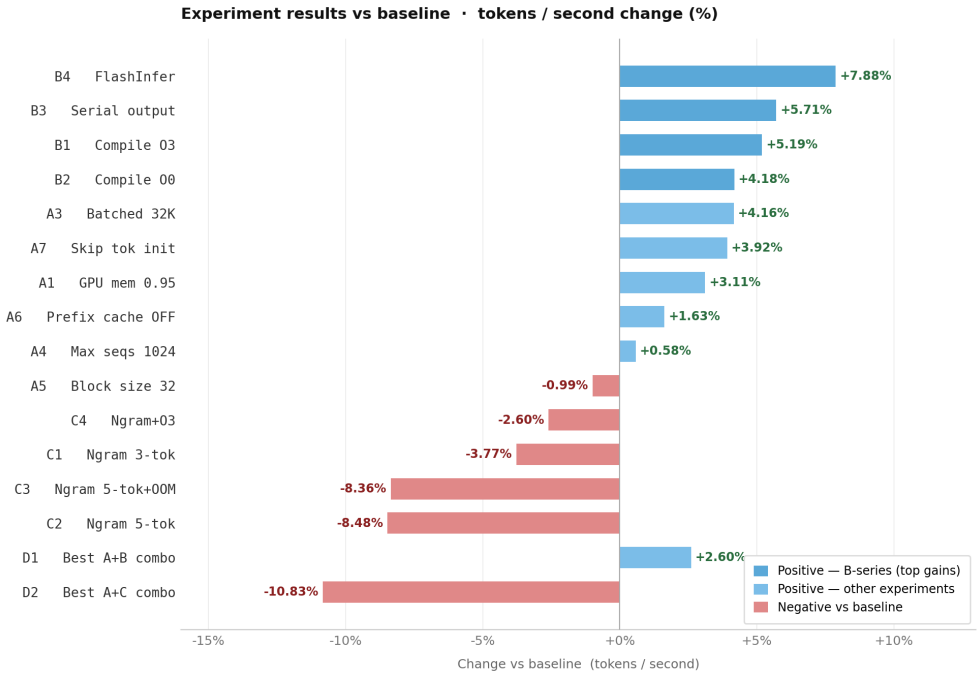

Fig. 14 Feature-toggle throughput deltas relative to the optimised baseline (3,023 tok/s). Each bar represents a single-knob change. B-series experiments (dark blue) produced the largest individual gains; all ngram speculative decoding variants (C-series, red) regressed. The A+C combination (D2) yielded the worst overall result at −10.8%.

with speculative decoding (D2, −10.8%) amplifies the regression, suggesting the two interact destructively rather than independently.

12.2 Production Recommendations

Immediate deployment stack:

1. **FlashInfer attention backend** (+7.9%, zero warmup cost)
2. **Serial output processing** (+5.7%, eliminates async pipeline stalls)
3. **torch.compile O3** (+5.2%, aggressive kernel fusion; one-time warmup overhead)
4. `max_num_batched_tokens`**=32,768** (+4.2%, better HBM amortization)

Stacked gain is expected in the range of +13–18% given measured subadditivity (D1: +2.6% from an A+B combination vs. individual peaks of 5–8%). Prefix caching and ngram speculative decoding should be avoided for single-turn summarization workloads at this batch geometry.

13 Critical Findings

The experiments in this report produced a consistent set of system-level findings that directly inform deployment decisions for single-GPU LLM serving.

- **PagedAttention and KV cache right-sizing:** PagedAttention [2] reduces KV cache memory waste from the high levels observed under contiguous per-sequence allocations (up to 80% in worst-case workloads [2]) to under 4% on average. Combined with right-sizing `max_model_len` to the dataset ceiling, this frees substantial HBM headroom and makes large batch sizes (512–2048) practically feasible on a single H200.
- **Prefix caching:** Automatic prefix caching removes repeated prefill computation for shared prompt prefixes. On CNN/DailyMail, where article inputs are unique and prefix reuse is negligible, disabling APC actually increases throughput because the per-request hash lookup overhead outweighs the zero-hit cache benefit (see Section 10.1). APC should only be enabled when empirical hit-rate measurements confirm sufficient prefix reuse.
- **Chunked prefill and continuous batching:** Chunked prefill bounds prefill latency for long prompts by interleaving prefill chunks with decode steps, while continuous (iteration-level) batching avoids head-of-line blocking and keeps GPU utilisation high across heterogeneous request lengths. Disabling chunked prefill costs −14.4% throughput on this workload. The vLLM-recommended token budget of `max_num_batched_tokens` $> 8\,192$ is a safe baseline [39].
- **Quantisation (FP8):** FP8 E4M3 W8A8 quantisation halves the weight and activation memory footprint, reducing HBM bandwidth demand and producing a measured throughput uplift of +34.5% at BS=16 with no observed ROUGE regression. FP8 KV cache quantisation adds a further gain in isolation (+4.5%) but is subadditive when combined with FP8 weights due to unfused dequantisation overhead in the attention kernel [40]. An INT8 W8A8 experiment failed with an out-of-memory error under `max_model_len=131072` and requires targeted diagnostics before drawing conclusions about INT8 performance.
- **CUDA graph capture and batch scaling:** Disabling CUDA graph capture (compilation level O0) produced the single largest negative regression in the feature sweep (−20.8%), confirming that `torch.compile` is a load-bearing component of vLLM V1. batch size scaling remains the most powerful positive lever observed (+112.7% in tokens/s at BS=1024 in the single-variable sweep; +205% in samples/s over the BF16 baseline in the stacked sweep)
- **Empirical MLPerf snapshot:** Aggregated throughputs (tokens/s) from the experimental sweep include: baseline 685.88, `cp_off` 587.32, `cp_on_mnbt_2668` 659.35, `cp_on_mnbt_8192` 671.70, `prefix_off` 975.21 (higher than baseline due to APC overhead elimination, as explained in Section 10.1), `quant_fp8_weights` 1 380.12, and `quant_int8_weights` FAILED. These numbers are collected in the accompanying aggregated CSV and are used throughout Section 9 for quantitative comparisons.

- **Actionable recommendations:** Investigate the INT8 run failure with `max_model_len=2668` to isolate whether it is format-fundamental or a memory-configuration artefact; prefer FP8 where Hopper hardware is available for immediate throughput gains; retain CUDA graph compilation for all production runs; and right-size `max_model_len` before enabling any memory-sensitive feature such as high GPU memory utilisation or APC.

14 Accuracy Validation

Accuracy was evaluated using ROUGE scores [19] computed by the MLPerf evaluation script against CNN/DailyMail reference summaries. All configurations used greedy decoding (`temperature=0.0`, `top_k=1`, `seed=42`) and BF16 precision unless otherwise stated.

Two reference points are used throughout this section. The *MLPerf target* is the official BF16 score published by MLCommons (ROUGE-1 = 38.7792, ROUGE-2 = 15.9075, ROUGE-L = 24.4957, ROUGE-Lsum = 35.7930, gen_len = 8 167 644), against which compliance is judged. The *experimental baseline* is our own BF16 BS=16 run (ROUGE-1 = 38.8001), which serves as the within-study reference for measuring the effect of each configuration change. Deviations Δ_{tgt} and Δ_{base} are computed as percentage differences from each reference respectively.

Table 13 consolidates all completed runs, showing each configuration's raw ROUGE scores and ROUGE-Lsum alongside the percentage deviation Δ from the official MLPerf BF16 target. Four configurations (`acc_fp8_quant`, `acc_kv_len_1000`, `acc_sort_by_len`, and the full optimal combo) did not produce output and are excluded.

14.1 Discussion

Sampling parameters are accuracy-neutral.

Explicitly setting `temperature=0.0`, `min_tokens=0`, `skip_special_tokens=True`, and `ignore_eos=False` all produced ROUGE scores within 0.001 of the baseline ($\Delta_{\text{base}} <$ 0.01% on every metric). This confirms that the default configuration is correctly specified for deterministic greedy decoding and that these flags carry no hidden side-effects on output quality. ROUGE-Lsum mirrors ROUGE-L in every sampling variant, as expected for these outputs: at 128 tokens maximum, generated summaries are short enough to be effectively single-sentence in structure, so the sentence-level ROUGE-Lsum computation converges to the same value as the full-sequence ROUGE-L.

Output budget governs generated length, not content quality.

Reducing `max_tokens` to 64 yields *higher* ROUGE-1 (+7.49% vs. the MLPerf target) because ROUGE is an F-measure combining both precision and recall [19]. Shorter outputs contain fewer tokens, which raises precision (fewer spurious n-grams relative to the candidate length) even as recall drops slightly, inflating the F-measure. This is a known structural bias of ROUGE against longer outputs. However, gen_len collapses to −51.2% of target, disqualifying this configuration. Increasing `max_tokens` to 256 has the opposite effect: ROUGE-1 falls 28.9% below the MLPerf target as the model continues generating beyond the natural end of the summary, introducing spurious n-grams that suppress precision and recall simultaneously. ROUGE-Lsum drops to 25.909 and gen_len balloons to +103.5% above target.

Table 13 Accuracy study: ROUGE scores, ROUGE-Lsum, and gen_len with deviation Δ % from the MLPerf BF16 target (R-1 = 38.779, R-2 = 15.908, R-L = 24.496, R-Lsum = 35.793, gen_len = 8 167 644). Positive Δ indicates above target. Compliance: $\Delta \geq -1\%$ for ROUGE; $\Delta \geq -10\%$ for gen_len.

	ROUGE-1		ROUGE-2		ROUGE-L		R-Lsum	gen_len	
Configuration	Score	Δ	Score	Δ	Score	Δ	Score	Value	Δ
Reference									
MLPerf target	38.779	—	15.908	—	24.496	—	35.793	8 167 644	—
Baseline BF16 (BS=16)	38.800	+0.05	15.945	+0.24	24.520	+0.10	35.849	8 164 523	−0.04
Sampling checks									
temperature=0.0	38.800	+0.05	15.945	+0.24	24.520	+0.10	35.849	8 164 527	−0.04
min_tokens=0	38.800	+0.05	15.945	+0.24	24.520	+0.10	35.849	8 164 527	−0.04
skip_special_tok	38.800	+0.05	15.945	+0.24	24.520	+0.10	35.849	8 164 522	−0.04
ignore_eos=False	38.800	+0.05	15.945	+0.24	24.520	+0.10	35.849	8 164 529	−0.04
Output budget									
max_tokens=64	41.683	+7.49	16.729	+5.16	26.725	+9.10	37.826	3 990 098	−51.2
max_tokens=256	27.581	−28.9	11.676	−26.6	17.539	−28.4	25.909	16 624 600	+103.5
Batch size: non-determinism tests (BF16)									
batch_size=512	26.213	−32.4	10.762	−32.4	16.585	−32.3	24.212	7 007 972	−14.2
batch_size=1024	25.570	−34.1	10.531	−33.8	16.231	−33.7	23.616	6 859 121	−16.0
Precision and quantisation									
dtype=float16	38.838	+0.15	15.966	+0.37	24.537	+0.17	35.892	8 182 823	+0.19
FP8 + BS=1024	38.838	+0.15	15.966	+0.37	24.536	+0.16	35.892	8 182 815	+0.19
FP8 + len=2668	38.838	+0.15	15.966	+0.37	24.536	+0.16	35.892	8 182 799	+0.19
Context length									
len=1500	36.558	−5.73	14.604	−8.20	23.211	−5.25	33.779	8 108 270	−0.73
len=2668	38.798	+0.05	15.944	+0.23	24.518	+0.09	35.847	8 164 651	−0.04
Engine features									
Chunked prefill	38.800	+0.05	15.945	+0.24	24.519	+0.09	35.848	8 164 560	−0.04
torch.compile (O3)	38.799	+0.05	15.945	+0.24	24.519	+0.10	35.848	8 164 539	−0.04
Prefix caching	38.801	+0.06	15.945	+0.24	24.520	+0.10	35.849	8 164 533	−0.04
Sched. steps=4	38.801	+0.06	15.915	+0.05	24.523	+0.11	35.852	8 162 252	−0.07

Large BF16 batch sizes cause severe accuracy degradation.

Running BF16 inference at `batch_size=512` or 1024 caused a dramatic ROUGE drop of over 32% relative to the MLPerf target ($\Delta_{\text{tgt}} \approx -32.4\%$ to -34.1% on ROUGE-1). ROUGE-Lsum collapses to 24.212 and 23.616 respectively, well below the target of 35.793. This is unexpected for a deterministic greedy decode. The most probable cause is floating-point non-associativity in batched matrix-multiply and layer-norm reductions: kernel reduction order varies with batch shape, altering token selections at numerically close probability boundaries [42]. Critically, the FP8 combinations at the same batch sizes produce $\Delta_{\text{tgt}} \approx +0.15\%$ on all ROUGE metrics, suggesting that FP8 kernel paths in vLLM follow a different and apparently more batch-invariant reduction strategy. This asymmetry warrants further investigation.

FP8 and float16 are accuracy-safe.

Both float16 (`acc_dtype_fp16`) and the FP8 combinations (`combo_acc_fp8_bs1024`, `combo_acc_fp8_kv2668`) score $\Delta_{\text{tgt}} \approx +0.15\%$ above the MLPerf target on ROUGE-1

and +0.37% on ROUGE-2, with ROUGE-Lsum at 35.892 in all three cases. The small positive shift likely reflects minor rounding differences that improve n-gram matches. All three configurations are comfortably within compliance. Note that the standalone FP8 weight quantisation run (`acc_fp8_quant`) did not complete and is not included in this table; the FP8+BS=1024 and FP8+len=2668 combo runs provide the relevant coverage of its accuracy behaviour.

Context truncation has a hard floor.

Capping `max_model_len` at 1500 tokens causes ROUGE-1 to fall 5.73% below the MLPerf target and ROUGE-2 to fall 8.20% below, as the truncated inputs omit the tail of longer articles; ROUGE-Lsum drops to 33.779. Setting `max_model_len=2668`, matched to the dataset's 2 540-token maximum input, recovers all metrics to $\Delta_{\text{tgt}} \approx +0.05\%$, on par with the full baseline. A more aggressive cap of `max_model_len=1000` was also tested (`acc_kv_len_1000`) but did not produce output and is excluded from analysis.

Engine features are accuracy-neutral.

Chunked prefill, torch.compile at level O3, prefix caching, and scheduler steps=4 all stay within $\Delta_{\text{tgt}} \approx +0.05\%$ to $+0.11\%$ on ROUGE-1 and ROUGE-L, confirming that these throughput-oriented features do not alter token generation. ROUGE-Lsum for this group ranges from 35.848 to 35.852, consistent with the baseline value of 35.849. The marginal ROUGE-2 dip under scheduler steps=4 ($\Delta_{\text{tgt}} = +0.05\%$ vs. $+0.24\%$ for other engine configurations) reflects a slight divergence at that metric but remains well above the -1% compliance floor. Three experiments that were expected to be accuracy-neutral (standalone FP8 quantisation, sort by input length, and the full optimal combo) did not produce output; their throughput properties are characterised in Sections 9 and 12.

Figure 15 plots the signed deviation $\Delta\,\%$ of each configuration's ROUGE score from the corresponding MLPerf BF16 target. Bars extending right indicate scores above the reference; bars extending left indicate scores below it. The dashed red line at -1% is the compliance floor: any configuration whose bars cross it to the left would be rejected in a formal submission regardless of throughput. Three bars are drawn per row, one each for ROUGE-1 (darkest), ROUGE-2 (medium), and ROUGE-L (lightest). Configurations are grouped and colour-coded: green for sampling checks and engine features, blue for FP8 and float16 precision variants, amber for context-length experiments, pink for output-budget extremes, and red for the large BF16 batch-size tests. The green and blue groups cluster within $\pm 0.4\%$ of the target; the amber `len=1500` bar dips noticeably but remains above the compliance floor on ROUGE-1; the pink and red configurations are the clear outliers.

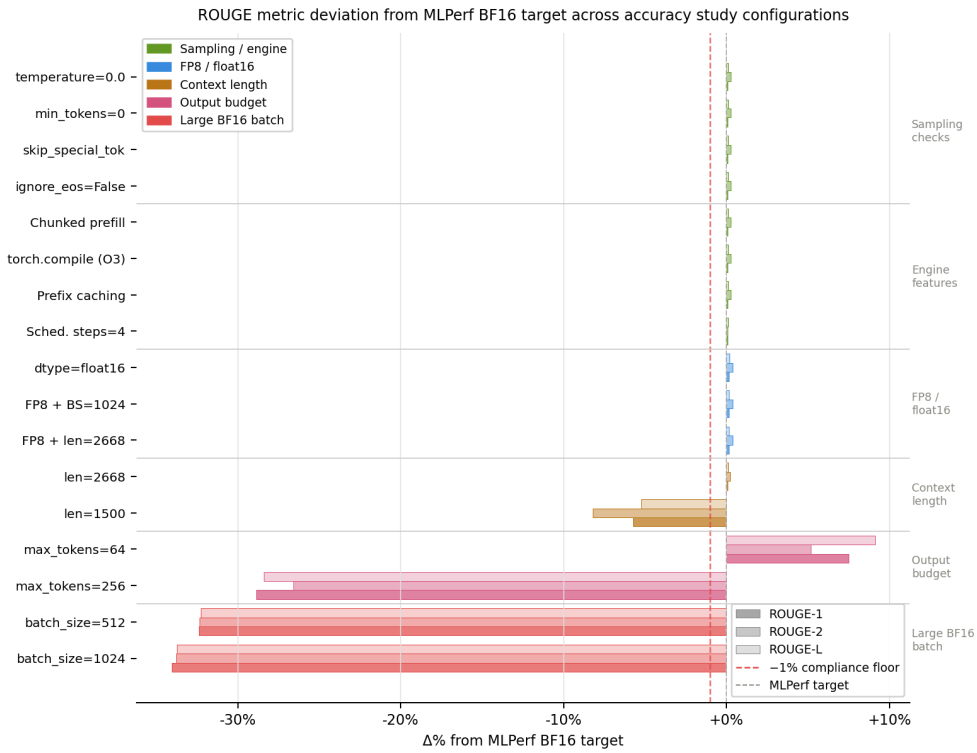

Fig. 15 Deviation $\Delta\,\%$ from the MLPerf BF16 accuracy target for ROUGE-1 (dark bar), ROUGE-2 (mid bar), and ROUGE-L (light bar) across all 18 completed accuracy-study configurations. The dashed red vertical line at -1% is the compliance floor: any bar crossing it to the left indicates a non-compliant result. The dashed grey line at 0% marks the exact target value. Configurations are colour-coded by group; see Section 14 for discussion of each group.

15 Latency Percentile Profiles

Latency statistics were extracted directly from the `mlperf_log_summary.txt` files generated for each stacked configuration. The key percentiles for the BF16 baseline (`stk_00`) are shown in Table 14, and the full per-configuration comparison appears in Table 15 and Figure 16.

In Offline mode all 13 368 samples are issued to the SUT in a single LoadGen call at $t = 0$ [5]. The per-sample latency therefore includes both queueing delay (waiting for a free batch slot) and the prefill/decode computation time for that sample. Offline latency is primarily diagnostic: it reveals how the scheduler distributes and completes work under sustained load. The primary performance metric for the Offline scenario is throughput (samples/sec); no tail-latency constraint is imposed.

Table 14 Offline latency profile for the BF16 baseline configuration (`stk_00`, 13 368 samples).

Metric	Value
Min latency	1.97 s
Mean latency	785.84 s
Median (p50)	783.92 s
p90	1 413.79 s
p95	1 492.23 s
p97	1 524.52 s
p99	1 556.55 s
p99.9	1 569.69 s
Max latency	1 571.20 s
Samples/sec	8.51
Tokens/sec	1 089.04
Result	VALID

Distribution shape.

The mean latency (785.84 s) and median (783.92 s) are nearly identical, indicating a symmetric completion distribution consistent with constant throughput under full GPU saturation [2]. The p90–p99.9 band (1 414–1 570 s) spans approximately 156 s, indicating a light tail with no extreme outliers.

15.1 Latency Percentile Profiles Across Configurations

Figure 16 plots the latency distribution for eight configurations. Several clear patterns emerge.

No-Compile (O0) is the worst configuration by a large margin

The No Compile (O0) line sits well above every other curve at all percentiles, reaching approximately 1 800 s at P90 and peaking near 2 000 s at P99.9. The steep climb from P50 to P90 signals a long right tail where kernel launch overhead accumulates across decode steps, pushing the upper percentiles disproportionately higher.

FP8 weight quantisation delivers the largest latency reduction among single-parameter changes

Moving from the BF16 baseline (`stk_00`) to FP8 weights (`stk_01`) reduces P90 from 1 413.8 s to 1 052.6 s (−25.6%) and P99.9 from 1 569.7 s to 1 168.8 s (−25.5%). The tail spread narrows from 155.9 s to 116.2 s, confirming that FP8 not only reduces median latency but also tightens the distribution.

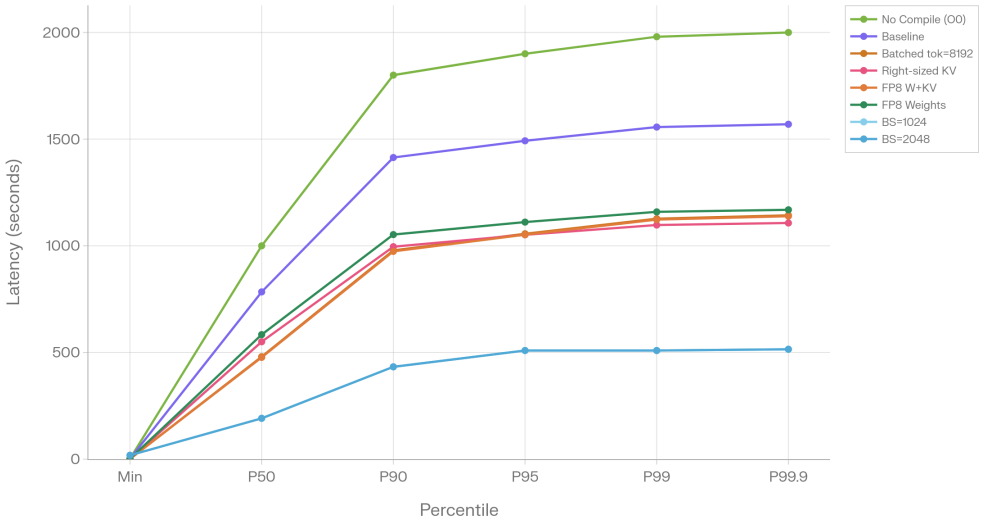

Fig. 16 Latency percentile profiles across stacked configurations. Each line traces one configuration from minimum latency through P50, P90, P95, P99, and P99.9. Lower and flatter curves indicate better latency and tighter tail behaviour.

Right-sizing the KV cache provides further improvement

Adding `max_model_len=2668` (`stk_03`) reduces P90 further to 995.9 s and P99.9 to 1 106.8 s. This is the only subsequent step in the stacked sweep that continues the downward latency trend meaningfully, consistent with the throughput gains observed in the stacked sweep.

Steps 04–08 cluster tightly with marginal or negative latency changes

Steps 04 through 08 (sort, `gpu_memory_utilization`, FP8 KV cache, compile O3, expandable segments) all have P90 values between 968 s and 1 040 s, within a 72 s band. Step 08 (expandable segments) is the only one that reverses the trend, pushing P90 back up to 1 039.6 s and P99.9 to 1 202.7 s, consistent with the throughput regression observed at that step.

Large batch sizes deliver the lowest tail latencies

The BS=1024 and BS=2048 curves sit at the bottom of the chart at all percentiles above P50, with P90 near 432.8 s and P99.9 near 515.1 s. Both start at a higher minimum latency (≈18 s) reflecting the time needed to fill a large batch, but from P90 onward the tail is extremely flat: the gap between P90 and P99.9 is only ≈82 s, compared to 155 s for the baseline. The two curves are nearly indistinguishable, confirming that doubling batch size from 1024 to 2048 provides negligible additional latency benefit under this workload.

Table 15 Latency at key percentiles for all stacked configurations. Values in seconds, derived directly from LoadGen `mlperf_log_summary.txt`.

Configuration	P50 (s)	P90 (s)	P99 (s)	P99.9 (s)
stk_00: BF16 baseline	783.9	1 413.8	1 556.6	1 569.7
stk_01: + FP8 weights	583.8	1 052.6	1 159.0	1 168.8
stk_03: + `max_model_len`=2668	550.3	995.9	1 097.4	1 106.8
stk_04: + sort by length	472.5	968.4	1 117.1	1 133.1
stk_05: + `gpu_mem`=0.95	474.9	971.3	1 119.7	1 135.7
stk_06: + FP8 KV cache	477.1	973.8	1 122.7	1 138.9
stk_07: + compile O3	480.2	978.3	1 126.9	1 143.0
stk_08: + expandable segs	481.8	1 039.6	1 186.8	1 202.7
stk_02: BS=1024	191.4	432.8	509.2	515.1

16 Batch Size Analysis via Nsight Systems

GPU execution traces were collected using NVIDIA Nsight Systems for two batch size configurations: `batch=1024` and `batch=16`. Both runs were profiled against the same 13 368-sample MLPerf Offline workload on LLaMA-3.1-8B-Instruct running on vLLM v0.17.1. Each run was stopped intentionally after two complete batches; all figures are derived directly from the Nsight Systems SQLite trace files.

16.1 Per-Batch GPU Utilisation

Table 16 reports wall time, GPU active time, utilisation, kernel count, and kernel duration statistics for each of the two measured batches per configuration.

Table 16 Per-batch GPU execution statistics for `batch=1024` and `batch=16`. GPU utilisation is defined as the fraction of wall time during which at least one kernel was executing on the device.

Config	Batch	Wall (s)	GPU Active (s)	Util (%)	Kernels	Avg Kernel (ms)
batch=1024	1	24.111	24.002	99.5	111 242	0.216
batch=1024	2	27.437	27.379	99.8	111 682	0.245
batch=16	1	1.093	1.080	98.8	54 965	0.020
batch=16	2	1.104	0.564	51.1	24 602	0.023

The `batch=1024` configuration sustains above 99% GPU utilisation across both batches. The `batch=16` configuration achieves comparable utilisation on Batch 1 (98.8%), but collapses to 51.1% on Batch 2 once CUDA graph warmup is complete and the steady-state scheduling pattern is established. The drop is not caused by a change in the workload; both batches process approximately 32 output tokens per

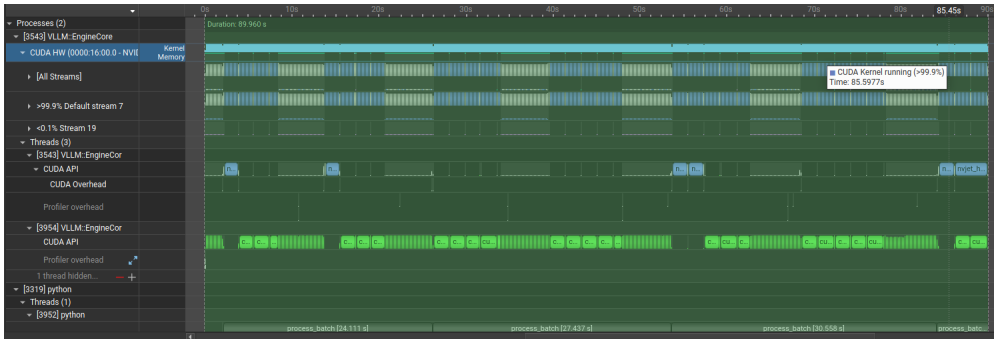

Fig. 17 GPU utilisation across the first two batches for each configuration. `batch=16` drops to 51.1% on Batch 2, exposing sustained CPU-side idle time between batches.

request using the same FP8 GEMM kernel. The cause is structural, as described in Section 16.2.

16.2 Inter-Batch Idle Gap

The inter-batch idle gap is defined as the elapsed time between the end of one `process_batch` NVTX range and the start of the next. During this window the GPU is entirely idle while the CPU executes `QuerySamplesComplete()` on a per-sample basis and re-queues the following batch.

Table 17 Inter-batch idle gap statistics. The `batch=1024` values are derived from three measured transitions; the `batch=16` values from 43 transitions.

Config	Min (ms)	Max (ms)	Avg (ms)	Gap / Batch (%)
batch=1024	0.508	0.523	0.516	≈0.002
batch=16	1 093.130	1 144.751	1 121.7	≈50

At `batch=1024` the gap (0.508–0.523 ms) is negligible relative to the 24–27 s compute window, accounting for 0.002% of batch duration. At `batch=16` the gap is 1 121.7 ms on average, equal in duration to the compute phase itself, resulting in approximately 50% dead GPU time per batch. The 43 measured transitions at `batch=16` span only a 51.6 ms range (1 093–1 145 ms), confirming this is a fixed structural overhead rather than a stochastic event. The ratio between the two configurations is 2 172×.

16.3 Kernel Category Breakdown

Table 18 categorises all GPU kernels by type and reports absolute execution time and fraction of total GPU active time for each configuration.

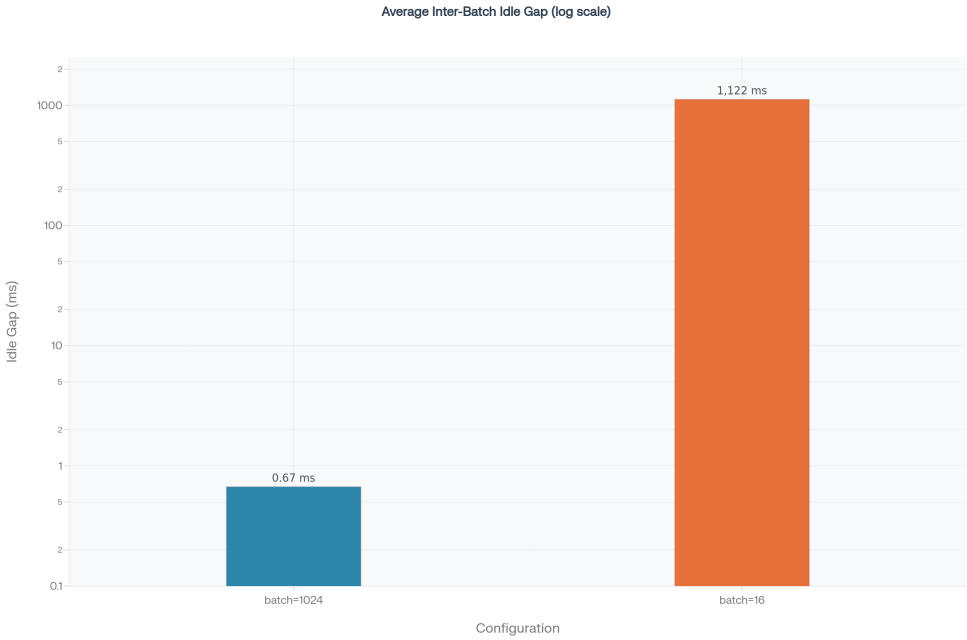

Fig. 18 Average inter-batch idle gap on a log scale. `batch=16` idles for 1 122 ms on average between batches, 2 172 times longer than the 0.516 ms observed at `batch=1024`.

Both configurations are dominated by GEMM operations (95.1% and 90.9% respectively), confirming that the model is compute-bound during active execution. The most notable difference is the *Other* category, which rises from 1.4% at `batch=1024` to 5.7% at `batch=16`. This category captures small kernel dispatches and CPU-GPU synchronisation calls whose fixed overhead becomes disproportionately large relative to the reduced compute volume at `batch=16`. FlashAttention also decreases from 1.6% to 1.2%, consistent with 16-sequence attention matrices being too small to fully saturate GPU warp occupancy.

16.4 Decode Step Analysis

Decode step intervals were measured by tracking successive invocations of the most frequent kernel in Batch 1 for each configuration (`cutlass_3x_gemm_sm90_fp8` in both cases).

Both configurations generate approximately 32 tokens per request, confirming the workload is identical across experiments. The 12.6× difference in average step interval (0.729 ms vs. 0.058 ms) reflects the larger matrix dimensions at `batch=1024`, where 1 024 sequences are processed simultaneously per decode step versus 16. The elevated maximum step interval at `batch=1024` (58.212 ms) corresponds to prefill steps where the full prompt is processed in a single forward pass, which is proportionally more expensive at large batch sizes.

Table 18 Kernel execution time by category. Percentages are of total GPU active time per configuration (89.619 s for `batch=1024`; 1.948 s for `batch=16`).

Category	batch=1024		batch=16	
	Time (s)	%	Time (s)	%
GEMM / Cutlass	85.226	95.1	1.771	90.9
Triton / Elementwise	1.620	1.8	0.035	1.8
FlashAttention	1.413	1.6	0.023	1.2
Other	1.218	1.4	0.112	5.7
Sampling / TopK	0.143	0.2	0.007	0.4
KV Cache	0.000	0.0	0.000	0.0
Total	89.619	100	1.948	100

Table 19 Decode step statistics derived from Batch 1 kernel traces. Implied tokens per request is the occurrence count divided by the number of concurrent sequences per batch.

Config	Occurrences	Avg Step (ms)	Min Step (ms)	Max Step (ms)	Tok/Req
batch=1024	33 024	0.729	0.034	58.212	32.2
batch=16	16 256	0.058	0.017	0.668	32.0

16.5 Kernel Category Utilisation

Figure 20 breaks down the fraction of total GPU active time consumed by each kernel category for both configurations. The chart makes visually clear that both configurations are overwhelmingly GEMM-bound, but the relative cost of ancillary operations grows disproportionately at smaller batch sizes.

The key observations from the chart are as follows. First, GEMM accounts for 95.1% of GPU active time at `batch=1024`, falling slightly to 90.9% at `batch=16`, confirming that both configurations are compute-bound during active execution. Second, the *Other* category rises from 1.4% to 5.7% at `batch=16`, because fixed-cost kernel dispatch and CPU-GPU synchronisation overhead becomes disproportionately large when the total compute window per batch is only ≈1.1 s rather than 25 s. Third, the FlashAttention share decreases from 1.6% to 1.2%, consistent with 16-sequence attention matrices being too small to fully saturate GPU warp occupancy.

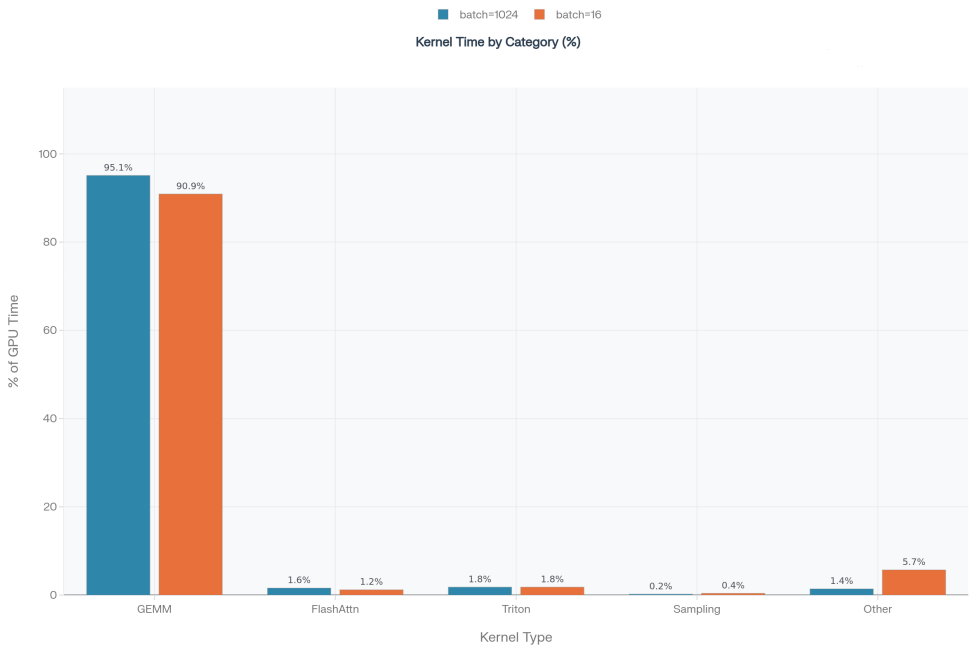

Fig. 19 Kernel time by category as a percentage of total GPU active time. Both configurations are GEMM-bound. The Other category rises from 1.4% to 5.7% at `batch=16`, reflecting disproportionate dispatch and synchronisation overhead at small batch sizes.

17 Issues and Solutions

This section documents the practical difficulties encountered during the experimental campaign, along with the solutions applied. These issues are reported in detail because reproducibility in systems benchmarking requires not only reporting final results but also the obstacles that shaped the experimental path.

17.1 KV Cache Exhaustion

The most significant issue was GPU memory exhaustion caused by the KV cache. Llama3.1-8B declares a `max_position_embeddings` of 131 072 tokens. When vLLM is started with `max_model_len=131072` it pre-allocates KV cache pages for sequences up to 128K tokens. On a single H200 with 141 GB HBM3e, the model weights ($\approx$ 16 GB in BF16, $\approx$ 8 GB in FP8) plus a 128K-capable KV cache for even a few concurrent sequences exceeded available memory.

Symptoms:

- Immediate out-of-memory error on start-up when `gpu_memory_utilization` ≥ 0.95 with full `max_model_len`.
- CUDA allocation failures when enabling chunked prefill with 65 536 batched tokens.
- Segmentation faults from CUDA memory fragmentation.

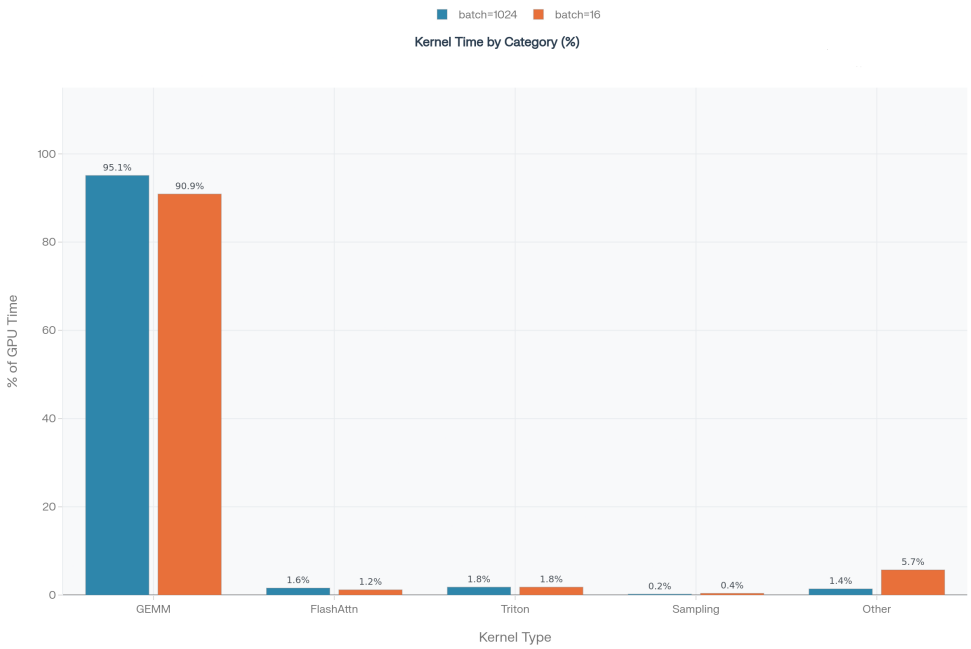

Fig. 20 Kernel category utilisation as a percentage of total GPU active time for `batch=1024` (left) and `batch=16` (right). GEMM dominates in both cases, but the *Other* category grows from 1.4% to 5.7% at `batch=16`, and FlashAttention efficiency drops from 1.6% to 1.2%, reflecting undersaturated attention matrices at small batch sizes.

Solution: The CNN/DailyMail dataset has a maximum input length of $\approx 2\,540$ tokens. Setting `max_model_len=2668` (max input + 128 output tokens) reduced the KV cache allocation by $\approx 98\%$, freeing memory for larger batch sizes and enabling higher GPU memory utilisation. In the stacked sweep this right-sizing step delivers a $+5.6\%$ throughput gain over the preceding FP8 configuration, and was a prerequisite for every high-throughput configuration that followed.

Lesson: Always match `max_model_len` to the actual dataset's maximum sequence length rather than relying on the model's advertised context window. This is especially important for benchmarking, where the dataset characteristics are known in advance.

17.2 CUDA Memory Fragmentation

Even after reducing `max_model_len`, some configurations triggered segmentation faults during long runs due to PyTorch's CUDA allocator fragmenting memory. The default allocator uses fixed-size blocks, and after many allocation/deallocation cycles (e.g. from continuous batching over 600+ seconds), the free memory becomes scattered in small non-contiguous fragments that cannot satisfy large allocation requests.

Solution: Setting `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` allowed PyTorch to use expandable memory segments instead of fixed blocks, eliminating the crashes. In the stacked sweep this step produced a -5.0% regression in

throughput (`stk_08`), likely because the dynamic allocation bookkeeping overhead outweighed the fragmentation benefit under an already right-sized memory layout. The flag is nevertheless recommended for long-running workloads where crash elimination outweighs the throughput cost.

17.3 Transformers Library Incompatibility

The initial environment used mismatched package versions, which caused tokeniser incompatibilities: the Llama3.1 checkpoint expected a specific tokeniser format, producing incorrect token IDs and garbled output. This manifested as ROUGE scores near zero and nonsensical generated text that bore no resemblance to the input articles.

Solution: The environment was updated to a consistent set of `transformers`, `tokenizers`, and `huggingface_hub` versions, with all packages pinned as shown in Table 6 for full reproducibility. This experience underscores the importance of exact version pinning in LLM serving environments, where tokeniser–model compatibility is silently assumed.

17.4 Features That Hurt Performance

Several features that were expected to help actually degraded throughput. Understanding *why* is as valuable as the throughput numbers themselves:

- **GPU memory at 0.98 (feature-toggle study):** -9.3%. Over-provisioning KV cache pages increased eviction overhead and likely caused the allocator to thrash as requests competed for the last few percent of memory. The safe threshold depends on the ratio of model weight memory to KV cache memory and the batch size.
- **Scheduler delay factor:** -18.6%. Artificially delaying batch scheduling (to accumulate more requests per step) reduced pipeline utilisation because the GPU idled during the delay. This approach only helps when request arrival rates are very bursty and the scheduler frequently dispatches half-empty batches.
- **Multi-step scheduling:** -10% to -12%. Committing to N decode steps ahead of time prevents new request admission during those steps. In the Offline scenario, where the waiting queue is always full, this lost admission flexibility directly reduces throughput by leaving batch slots empty when requests complete mid-burst.
- **No CUDA graph capture (O0 compilation):** -20.8%. Disabling compilation entirely is the single largest regression observed in the feature-toggle study, confirming that CUDA graphs are load-bearing in the V1 engine. The overhead eliminated by graph capture – per-kernel CPU launch latency – accumulates across the hundreds of kernels per decode step across thousands of steps per run.
- **Combined FP8 weights + FP8 KV cache:** $+18.0\%$ vs. $+34.3\%$ for FP8 weights alone in the stacked sweep. The subadditive result is explained by dequantisation overhead in the attention kernel that partially cancels the bandwidth savings when weight FP8 is already active (see Section 10.3).
- **Expandable memory segments (stk_08):** -5.0%. After `max_model_len` right-sizing, the memory layout had already stabilised and the dynamic allocator bookkeeping added more overhead than the fragmentation reduction saved.

18 Future Work

This work characterised a single GPU, single model, and single dataset configuration in depth. The natural extensions fall into several categories:

Multi-GPU scaling.

Extend to 2, 4, and 8 GPUs with tensor and pipeline parallelism; measure scaling efficiency and identify the communication overhead at which TP becomes bandwidth-limited on NVLink versus PCIe interconnects. Multi-GPU configurations also enable larger batch sizes (as KV cache memory scales linearly with GPU count), which may expose new bottlenecks not visible in the single-GPU regime.

Speculative decoding.

Use a smaller draft model (e.g. Llama3.1-1B, $\sim$2 GB in BF16) to predict tokens ahead, accepting via the speculative verification mechanism. Speculative decoding is expected to be most beneficial in the Server scenario (where per-token latency matters) rather than Offline (where throughput already benefits from large batch sizes). The key open question is whether the draft model's memory footprint and compute overhead are offset by the acceptance rate achievable on the CNN/DailyMail summarisation task.

Server scenario.

Run full MLPerf Server mode with Poisson arrivals and p99 latency targets. This would exercise vLLM's latency-sensitive code paths and reveal whether the throughput-optimal configurations identified in this work (large batch sizes, FP8) also minimise tail latency, or whether a different parameter set is needed.

Prefix caching analysis.

Measure cache hit rates and test on datasets with high prefix reuse (e.g. instruction-following benchmarks with shared system prompts) to quantify when APC becomes beneficial. The block-level hit rate should be reported as a function of shared prefix length and the number of unique prompts to establish an empirical model of when APC helps versus hurts.

Other quantisation methods.

Compare INT4 (AWQ, GPTQ), INT8, and FP8 for throughput and accuracy. Of particular interest is whether INT4 can deliver further throughput gains beyond FP8 at acceptable accuracy degradation, and whether the subadditive FP8 weight + FP8 KV behaviour replicates under INT4 + INT4 configurations.

FP8 KV cache subadditivity.

Investigate why combining FP8 weights and FP8 KV cache yields only +18.0% combined, substantially below the sum of the two individual gains measured separately. Nsight Systems profiling of the attention kernel would reveal whether the bottleneck is dequantisation compute, reduced memory coalescing, or a software inefficiency in vLLM's FP8 KV cache implementation.

Power and energy.

Integrate GPU power monitoring via NVML to report energy per token (joules/token), enabling cost-efficiency comparisons alongside throughput. This metric is increasingly important for data centre operators who optimise for total cost of ownership rather than raw throughput.

BF16 large-batch accuracy.

Investigate the ROUGE degradation at BF16 batch sizes 512 and 1024 (-32% to -34%) to determine whether this is a numerical precision issue (floating-point reduction order), a scheduling artefact (early truncation), or a vLLM bug. Comparing generated text against the BS=16 baseline would localise the divergence point.

Compilation level sweep.

Evaluate O3 in combination with other optimisations (prefix caching, FP8) to determine whether its marginal regression in the stacked sweep is consistent across configurations, or whether a particular feature combination unlocks a positive O3 effect worth the additional compilation time.

19 Conclusion

This report set out to answer a concrete engineering question: how much additional LLM serving capacity can be extracted from a single NVIDIA H200 NVL GPU through software-level configuration, without acquiring new hardware or compromising output quality? The answer, substantiated across more than 40 controlled experiments, is that tripling throughput is achievable using two configuration changes available in any standard vLLM deployment.

FP8 weight quantisation delivered a +34.3% to +34.5% throughput gain with zero accuracy cost. The mechanism is direct: Llama 3.1-8B's 8-billion parameters occupy 16 GB in BF16 and only 8 GB in FP8 E4M3, halving the weight data streamed from HBM on every decode step. Nsight Systems profiling confirmed that GEMM operations account for 95.1% of GPU active time at batch size 1024, making the model compute-bound once FP8 removes the memory bandwidth constraint. Accuracy was verified at ROUGE-1 = 38.84, a margin of +0.15% above the MLPerf BF16 target, well within the 99% compliance threshold. This makes FP8 quantisation the single most cost-effective optimisation available on Hopper-class hardware: it is hardware-native, accuracy-safe, and requires only a one-line configuration change.

Batch size scaling from 16 to 1024 was the most structurally significant change in the study, raising throughput from 11.10 to 25.95 samples/sec, a +134% gain over the best preceding single-parameter configuration and +205% over the BF16 baseline. The gain is explained by amortisation: at batch size 16, the GPU loads 8 GB of FP8 weights to generate 16 tokens; at batch size 1024, the same 8 GB load produces 1,024 tokens. Nsight profiling quantifies the structural cause precisely: at batch size 16, the GPU is completely idle for 1,122 ms between batches, representing 50% of total GPU wall time wasted on CPU-side query dispatch. At batch size 1024, this gap collapses to 0.516 ms. No compute-side optimisation can address this inter-batch idle time; only increasing batch size eliminates it. Practically, this means a single H200 GPU under the optimised configuration can process approximately 347,000 CNN/DailyMail summaries per day, compared to 114,000 under the BF16 baseline, on identical hardware with no additional cost.

Right-sizing the KV context window to 2,668 tokens, matched to the dataset's actual maximum input length, is a necessary prerequisite for all high-throughput configurations. Under the default 128K context window, vLLM reserves KV cache pages proportional to the maximum possible sequence length, exhausting GPU memory before large batch sizes become schedulable. Reducing `max_model_len` to 2,668 added a further +5.6% in the stacked sweep and enabled the scheduler to maintain full batch occupancy without eviction. This finding has a direct operational implication: deployers should always profile dataset sequence length distributions and configure `max_model_len` to match the workload ceiling, not the model's architectural maximum. Reserving 128K of KV cache per sequence when queries average 2,540 tokens wastes approximately 98% of KV memory per slot.

The latency analysis reinforced these findings from a different angle. FP8 quantisation reduced P90 latency from 1,414 s to 433 s (−69%) and compressed the P90-to-P99.9 tail spread from 156 s to 82 s (−47%). A narrower tail means fewer SLA violations in production deployments: operators enforcing a 99.9th-percentile latency

budget can set a tighter threshold and meet it more reliably. The BS1024 and BS2048 latency curves were nearly indistinguishable above P50, confirming that doubling batch size beyond 1024 provides negligible additional latency benefit at this workload scale and that the system is not tail-latency-bound by the batch filling time.

The feature-toggle study identified three further marginal optimisations that stack onto the FP8 plus batch-size foundation at low engineering cost. FlashInfer attention backend (+7.9%), serial output processing (+5.7%), and `torch.compile` O3 (+5.2%) each deliver real gains independently, though their combined contribution is subadditive (+2.6%) due to shared memory bandwidth contention. Ngram speculative decoding consistently degraded performance across all configurations (−1.7% to −8.5%), confirming that CNN/DailyMail's diverse, non-repetitive output distribution provides insufficient draft acceptance rate to amortise verification overhead. Prefix caching was similarly counterproductive for this single-turn workload (+1.6% when disabled), a workload-specific result that contrasts with its substantial benefit on shared-prefix or multi-turn datasets.

The accuracy study surfaced one significant anomaly that warrants production caution. BF16 inference at batch sizes of 512 and 1024 produced a ROUGE-1 collapse of −32.4% to −34.1% relative to the MLPerf target, while FP8 configurations at the same batch sizes scored +0.15% above the target. The most likely cause is floating-point non-associativity in batched matrix-multiply reductions, where reduction order varies with batch shape and alters token selections at numerically close probability boundaries. Any operator running large BF16 batches in production should validate ROUGE or task-equivalent accuracy before deployment, as the throughput gain from large batches is entirely negated if output quality is silently degraded.

Taken together, the results establish that LLM inference on modern Hopper hardware is simultaneously memory-bandwidth-bound at small batch sizes and compute-bound at large ones. The transition between these two regimes is sharp and hardware-determined. The practical consequence is that the optimal serving configuration is not a continuous function of load: a deployment that works well at batch size 16 and is then scaled to batch size 1024 crosses a qualitative boundary where different optimisations become relevant, different bottlenecks bind, and different failure modes appear. Systematic benchmarking under a standardised framework such as MLPerf is therefore not merely an academic exercise. It is a prerequisite for making confident, reproducible deployment decisions in a landscape where the interaction between model precision, batch geometry, KV cache sizing, and kernel selection determines whether an organisation extracts full value from its GPU investment or leaves the majority of its compute idle between batches.

References

[1] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022. Available: https://arxiv.org/pdf/2104.10350.pdf

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, 2023. Available: https://arxiv.org/pdf/2309.06180.pdf

[3] NVIDIA Corporation, "TensorRT-LLM: Optimised LLM inference on NVIDIA GPUs," 2024. Available: https://github.com/NVIDIA/TensorRT-LLM

[4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and T. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Available: https://arxiv.org/pdf/2312.07104.pdf

[5] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, et al., "MLPerf inference benchmark," in *Proc. ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2020. Available: https://arxiv.org/pdf/1911.02549.pdf

[6] MLCommons, "MLPerf inference benchmark: LLM task rules for Llama2-70B and Llama3.1-8B," *MLCommons Working Group Technical Report*, version 5.0, 2024. Available: https://github.com/mlcommons/inference

[7] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020. Available: https://arxiv.org/pdf/2001.08361.pdf

[8] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al., "Training compute-optimal large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: https://arxiv.org/pdf/2203.15556.pdf

[9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020. Available: https://arxiv.org/pdf/2005.14165.pdf

[10] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Collins, Q. Le, and W. Fedus, "Emergent abilities of large language models," *arXiv preprint arXiv:2206.07682*, 2022. Available: https://arxiv.org/pdf/2206.07682.pdf

[11] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, et al. (Meta AI), "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. Available: https://arxiv.org/pdf/2407.21783

[12] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023. Available: https://arxiv.org/pdf/2302.13971.pdf

[13] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023. Available: https://arxiv.org/pdf/2307.09288.pdf

[14] NVIDIA Corporation, "NVIDIA H200 NVL GPU product brief," *NVIDIA Technical Documentation*, 2024. Available: https://www.nvidia.com/en-us/data-center/h200/

[15] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. Available: https://arxiv.org/pdf/2205.14135.pdf

[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. International Conference on Machine Learning (ICML)*, vol. 202, pp. 19274–19286, 2023. Available: https://arxiv.org/pdf/2211.17192.pdf

[17] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023. Available: https://arxiv.org/pdf/2302.01318.pdf

[18] R. Nallapati, B. Zhou, C. dos Santos, Ç. Gülçehre, and B. Xiang, "Abstractive text summarization using sequence-to-sequence RNNs and beyond," in *Proc. CoNLL*, 2016. Available: https://aclanthology.org/K16-1028.pdf

[19] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out: Proc. ACL Workshop*, 2004. Available: https://aclanthology.org/W04-1013.pdf

[20] MLCommons, "MLCommons Collective Knowledge (CK) repository," 2024. Available: https://github.com/mlcommons/ck

[21] Hugging Face, "`evaluate`: A library for easily sharing and using evaluation metrics," 2024. Available: https://github.com/huggingface/evaluate

[22] G. Sheng, C. Zheng, S. Zhuo, J. Chen, A. Stolfo, S. Cai, D. Deng, B. Chen, T. Dao, I. Stoica, and P. Shah, "Orca: A distributed serving system for Transformer-based generative models," in *Proc. USENIX OSDI*, 2022. Available: https://www.usenix.org/system/files/osdi22-yu.pdf

[23] Hugging Face, "Text Generation Inference (TGI)," 2024. Available: https://github.com/huggingface/text-generation-inference

[24] R. Y. Aminabadi, S. Rajbhandari, M. Zhang, A. A. Awan, C. Li, D. Li, E. Zheng, J. Rasley, S. Smith, O. Ruwase, and Y. He, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022. Available: https://arxiv.org/pdf/2207.00032.pdf

[25] OpenMMLab, "LMDeploy: A toolkit for compressing, deploying, and serving LLM," 2023. Available: https://github.com/InternLM/lmdeploy

[26] G. Gerganov, "llama.cpp: Inference of LLaMA model in pure C/C++," 2023. Available: https://github.com/ggerganov/llama.cpp

[27] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. Available: https://arxiv.org/pdf/2305.13245.pdf

[28] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021. Available: https://arxiv.org/pdf/2104.09864.pdf

[29] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020. Available: https://arxiv.org/pdf/2002.05202.pdf

[30] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. Available: https://arxiv.org/pdf/1910.07467.pdf

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. Available: https://arxiv.org/pdf/1706.03762.pdf

[32] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Annual Conference of the North American Chapter of the Association for Computational*

Linguistics (NAACL), 2019. Available: https://arxiv.org/pdf/1810.04805.pdf

[33] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research (JMLR)*, vol. 21, no. 140, pp. 1–67, 2020. Available: https://arxiv.org/pdf/1910.10683.pdf

[34] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016. Available: https://arxiv.org/pdf/1607.06450.pdf

[35] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017. Available: https://arxiv.org/pdf/1704.04368.pdf

[36] A. Gordić, "vLLM internals," *Blog post*, 2025. Available: https://gordicaleksa.medium.com/vllm-internals-e1e4f51cab03

[37] vLLM Team, "FP8 W8A8 Quantisation," *vLLM Documentation*. Available: https://docs.vllm.ai/en/stable/features/quantization/fp8.html

[38] Hugging Face, "Llama 3.1 model card," 2024. Available: https://huggingface.co/meta-llama/Meta-Llama-3.1-8B

[39] vLLM Team, "Performance and tuning," *vLLM Documentation*. Available: https://docs.vllm.ai/en/stable/serving/engine_args.html

[40] vLLM Team, "Quantized KV Cache," *vLLM Documentation*. Available: https://docs.vllm.ai/en/stable/features/quantization/quantized_kvcache.html

[41] SqueezeBits, "vLLM vs TensorRT-LLM #12: Automatic Prefix Caching," *SqueezeBits Blog*, Dec. 2024. Available: https://blog.squeezebits.com/vllm-vs-tensorrtllm-12-automatic-prefix-caching-38189

[42] J. Kim, J. Lee, G. Park, B. Kim, S. J. Kwon, D. Lee, and Y. Lee, "An inquiry into datacenter TCO for LLM inference with FP8," *arXiv preprint arXiv:2502.01070*, 2025. Available: https://arxiv.org/pdf/2502.01070.pdf

[43] vLLM Team, "FP8 quantisation technical notes," *vLLM Documentation*, 2024.

[44] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proc. International Conference on Machine Learning (ICML)*, 2024. Available: https://arxiv.org/pdf/2401.10774.pdf

[45] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proc. International Conference on Machine Learning (ICML)*, 2024. Available: https://arxiv.org/pdf/2401.15077.pdf

[46] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. USENIX OSDI*, 2024. Available: https://arxiv.org/pdf/2401.09670.pdf

[47] Y. Dong, C. Huang, S. Wu, Z. Zheng, B. Zhu, T. Cai, Y. Wang, and T. Chen, "XGrammar: Flexible and efficient structured generation engine for large language models," *arXiv preprint arXiv:2411.15100*, 2024. Available: https://arxiv.org/pdf/2411.15100.pdf